QueuePulse – Engineering Blueprint

Oct 20, 2026 · @MD HUJAIFA AZAD

1. Project Understanding

QueuePulse removes the "which number is running?" uncertainty at chambers, diagnostic centers, banks and service centers. It shows the live serial and a computed waiting time on the visitor's own phone, with no app and no login.

Item	Summary
Core problem	Visitors wait 2–3 hours after booking because the current serial is invisible; receptionists answer the same call repeatedly.
Target users	Visitors (no account), receptionists/staff, platform administrator.
Value proposition	Stay home until your turn is near; staff advance the queue with one click.
Essential features	Token + QR, Next/Skip controller, live view, moving-average ETA, near-turn alert.
Constraints	3-person student team (Database; Frontend + Project management; Backend/API), 10-week timeline, free-tier hosting assumed.
Existing code	None stated; treated as a new repository.

Measurable success criteria (targets to validate, not yet measured)

- Serial change appears on 95% of connected visitor screens within 2 seconds, with 100 viewers on one queue.
- A staff "Next" action needs 1 click and the server responds within 1 second.
- Visitor live page shows its first useful content within 3 seconds on a mid-range phone over 4G.
- ETA accuracy is reported from a simulated queue test; target mean absolute error of 20% or less when service times are stable.

2. MVP Scope

The MVP is one complete loop: staff issue a token, advance the queue, and the visitor sees live position and ETA. Everything else waits.

Priority	Feature	Note
Must	Staff sign-in, role-based access	Accounts created by admin
Must	Organizations, staff and queue setup (admin / staff)	Minimal forms
Must	Issue token, QR slip, public live link	Sequential number per queue per day
Must	1-click Next / Skip controller	Atomic, safe for two staff clicking together
Must	Visitor live view (no login)	Realtime plus polling fallback
Must	Moving-average wait-time (last 5)	Rules for skips, breaks, cold start in FR-11
Must	Near-turn alert at 3 ahead	In-page alert plus Web Push
Must	Basic reports	Tokens, average wait, average service time
Should	SMS alert through a gateway	Needs phone number and per-SMS cost; behind a provider adapter
Should	Pause / resume queue, recall skipped token	Real receptionist needs, small effort
Should	Peak-hour report, CSV export	After core reports work
Excluded	Payments, native app, ML prediction, hospital/bank integration	As in the proposal
Excluded	Visitor accounts, online booking, WhatsApp/voice alerts	Future enhancements

3. Assumptions, Contradictions, Risks and Open Decisions

Contradictions and weak assumptions found in the proposal

ID	Issue	Resolution proposed
C-01	"Registration" for staff, but the admin manages staff accounts.	No public sign-up. Admin creates staff; staff log in and reset password.
C-02	"Zero-app browser alert" is unreliable on iPhone: Web Push there generally needs the page added to the Home Screen (verify against current Apple documentation).	In-page alert (banner, vibration) works everywhere while the page is open; Web Push is best effort; SMS is the reliable fallback.
C-03	"Token slip with QR" assumes a printer.	Show the QR on screen and print through the browser print dialog; no printer driver work.
C-04	Average of the last 5 services breaks on lunch breaks, skips and the first tokens of the day.	Defined in FR-11: exclude skipped and over-long durations, default minutes per queue.
C-05	"Administrator" is both platform owner and clinic manager.	Two roles: `admin` (platform) and `staff` (bound to one organization).

Assumptions

ID	Assumption
A-01	Queue day resets at midnight Asia/Dhaka.
A-02	Visitor phone number is optional and deleted 7 days after the token day.
A-03	Service time = time between a token being called and the next token being called. Staff must press Next honestly for the ETA to be meaningful.
A-04	Free tiers of Vercel and Supabase are enough for a capstone demo. Their current limits (realtime connections, project pausing on inactivity) are not verified here and must be checked before deployment.
A-05	Pilot data from a real chamber is not guaranteed; accuracy will be shown with a simulation.

Risks

ID	Risk	Probability	Impact	Mitigation	Phase
R-01	Two staff click Next together and a token is skipped or duplicated	Medium	High	One Postgres function with a row lock; concurrency test	DB / API
R-02	Realtime drops on weak mobile networks	High	Medium	Auto-reconnect plus 10-second polling fallback	Frontend
R-03	Web Push not delivered (iOS, blocked permission)	High	Medium	In-page alert, SMS option, clear on-screen message	Notifications
R-04	SMS cost or gateway approval delays	Medium	Medium	Provider adapter with mock; SMS marked Should	Notifications
R-05	Public live page leaks data	Low	High	Page shows only numbers; unguessable IDs; RLS	Security
R-06	Inaccurate ETA reduces trust	Medium	Medium	Show "about", rules in FR-11, simulation test	Backend
R-07	Scope growth in a 10-week, 3-person project	Medium	High	Fixed MVP table; Should items only after Must pass tests	All

Open decisions (default applies if you do not answer)

ID	Question	Default
D-01	Visitor page language	Bangla default, English toggle; staff UI in English
D-02	SMS gateway provider	Adapter interface, mock provider until you choose one
D-03	Can a skipped token be recalled?	Yes, once, by staff (Should)

4. Technology Direction

Keep the stack from the proposal: Next.js with TypeScript, Supabase (PostgreSQL, Auth, Realtime, Row Level Security), deployed on Vercel. It gives realtime, auth and a database without running a server, which suits 3 students and 10 weeks. A modular monolith (one Next.js app plus one database) is enough; no microservices or queues.

ID	Decision	Rationale	Alternative considered
DEC-01	Modular monolith in Next.js (App Router)	One deploy, one codebase	Separate Node API: more work, no benefit at this size
DEC-02	Queue changes run inside Postgres functions	Row lock prevents race conditions (R-01)	Logic in route handlers: unsafe under concurrent clicks
DEC-03	Realtime via `Broadcast from Database: a trigger on queue_state sends counters to a public topic per queue (revised in Section 14)`	Visitors subscribe to numbers only, no personal data	Postgres Changes: needs anonymous read on a table; subscribing to tokens would expose phone numbers
DEC-04	Polling fallback every 10 seconds	Covers dropped sockets (R-02)	Realtime only: fails silently
DEC-05	Notification outbox table	A failed push/SMS is retried, not lost	Send inline only: loses alerts on a crash

ID	Decision	Rationale	Alternative considered
DEC-06	Added: Zod (validation), Tailwind CSS, Playwright (end-to-end)	Validation at trust boundaries; fast UI; critical journey tests	Not in your proposal; remove if you prefer
DEC-07	Testing: Jest for logic, Postman collection for API, Playwright for end-to-end	Keeps your Jest and Postman choice	Vitest: equivalent, but proposal says Jest

5. Product Requirements Document (PRD)

Personas

- **Rahim, visitor:** booked a doctor, wants to stay home until his turn is near. Uses an Android phone, low patience for apps.
- **Nasrin, receptionist:** handles 60–100 patients a day, wants one button and no repeated phone calls.
- **Platform admin:** creates organizations and staff accounts, checks that the system runs.

Primary use cases

- UC-01: Staff issue a token; the visitor scans the QR and opens the live page.
- UC-02: Staff press Next; every viewer's screen and ETA update.
- UC-03: The visitor is alerted when 3 people are ahead.
- UC-04: Admin or staff review daily counts and average waiting time.

Functional requirements

ID	Requirement	Acceptance criteria	Priority
FR-01	Staff and admin sign in with email and password.	Wrong credentials give one generic error; logout ends the session.	Must
FR-02	Password reset by email.	Reset link works once and expires.	Must
FR-03	Admin manages organizations (create, edit, deactivate).	Deactivated organization's staff cannot operate queues.	Must

ID	Requirement	Acceptance criteria	Priority
FR-04	Admin creates and deactivates staff accounts bound to one organization.	Staff see only their organization's data.	Must
FR-05	Staff create and configure queues (name, default minutes per patient, active flag).	Only own organization's queues are visible.	Must
FR-06	Staff issue a token with an optional visitor phone number.	Numbers are sequential per queue per day; 20 simultaneous requests give 20 unique numbers.	Must
FR-07	Token slip shows queue name, number, date and QR.	QR opens the token's live page; slip prints from the browser.	Must
FR-08	Next: current token becomes `completed`, next `waiting` token becomes `serving`.	One click; two staff clicking together never skip or repeat a token.	Must
FR-09	Skip: current token becomes `skipped`, next token is called.	Skipped token never counts in the average.	Must
FR-10	Visitor live page needs no login and shows serving number, own number, people ahead, ETA, status.	Updates within 2 seconds; reconnects after network loss.	Must
FR-11	ETA = people ahead × average service minutes. People ahead counts the serving token plus `waiting` tokens with a lower number. Average = mean of the last 5 `completed` durations; fewer than 5 uses those available; none uses the queue default; durations above 60 minutes or under 20 seconds are excluded (D-04).	Unit tests cover each case; example 6 people × 7 min = 42 min.	Must
FR-12	Near-turn alert fires once per token when people ahead first becomes 3 or fewer.	In-page alert and Web Push to subscribed visitors; no duplicate alerts after refresh.	Must

ID	Requirement	Acceptance criteria	Priority
FR-13	Token states: `waiting`, `serving`, `completed`, `skipped`, `cancelled`, expired (set by the system at day end). Staff can cancel a waiting token.	Visitor sees a clear message for each state.	Must
FR-14	Staff dashboard shows current token, next 5 waiting, counts, current average.	Matches database after every action.	Must
FR-15	Reports by queue and date range: tokens issued, completed, skipped, average wait (issued to called), average service.	Figures match a seeded test dataset.	Must
FR-16	Admin dashboard shows organizations, active queues, tokens today.	Admin only.	Must
FR-17	SMS alert for tokens with a phone number.	Sent once through the provider adapter; failures are retried and logged.	Should
FR-18	Pause and resume a queue; recall one skipped token.	Paused queue shows "paused" to visitors; ETA is not counted during pause.	Should
FR-19	Peak-hour report and CSV export.	Hourly counts match test data.	Should

Non-functional requirements

ID	Requirement	Target
NFR-01	Realtime latency	95% of updates in 2 seconds with 100 viewers per queue
NFR-02	Page load	Visitor page first content in 3 seconds, mid-range phone, 4G
NFR-03	Degraded mode	If the system is down, staff can continue manually; queue state recovers when it returns

ID	Requirement	Target
NFR-04	Responsive	Works from 360 px width; staff dashboard usable on tablet and desktop
NFR-05	Accessibility	Labels, focus states and colour contrast to WCAG 2.1 AA basics; state not shown by colour alone
NFR-06	Browser support	Last 2 versions of Chrome, Safari, Firefox, Edge
NFR-07	Privacy	Phone numbers never public; deleted 7 days after token day (A-02)
NFR-08	Maintainability	TypeScript strict mode, lint clean, 70% or more coverage on business-logic modules
NFR-09	Time handling	Queue day and reports use Asia/Dhaka; timestamps stored in UTC

Security controls (SEC-xx) are specified in Part 2.

Constraints and dependencies: 10 weeks, 3 students, free-tier hosting (A-04), SMS gateway account (D-02), HTTPS domain for Web Push.

6. Technical Requirements Document (TRD)

Responsibilities by layer

Layer	Technology	Responsibility
Frontend	Next.js, React, TypeScript, Tailwind, qrcode.react	Staff dashboard, admin pages, public live page, token slip
Backend	Next.js route handlers	Auth-checked staff actions, notification sending, reports
Database	PostgreSQL on Supabase	Data, constraints, queue functions (lock, numbering), Row Level Security
Auth	Supabase Auth	Email and password sessions for staff and admin

Layer	Technology	Responsibility
Realtime	Supabase Realtime Broadcast from Database, one public topic per queue	Live serial to visitor screens
Notifications	Web Push (VAPID keys) and SMS adapter	Near-turn alerts

Authentication and authorization

- Roles: `admin` (platform) and `staff` (one organization). Visitors are anonymous.
- Authorization is enforced by Row Level Security and checked again in route handlers; the frontend never decides access.
- Visitor access is by a random, unguessable public code in the QR link, not by login.

APIs and integrations (high level; full contracts in Part 3)

- Staff and admin actions through authenticated route handlers that call Postgres functions with the user's own session.
- Public read of one token's view through a narrow Postgres function that returns only numbers and status.
- Web Push using the standard Push API; SMS through one provider behind an adapter interface.

Environments

Environment	Setup
Local	Next.js dev server plus Supabase CLI (Docker) with seed data
Production	Vercel plus a Supabase cloud project. A staging project is optional if free limits allow

Configuration and secrets

Variable	Where	Exposure
`NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`	Browser and server	Public by design; safe only because of RLS

Variable	Where	Exposure
`SUPABASE_SERVICE_ROLE_KEY`	Server only	Never in client code; used only for outbox processing, push registration, staff-account creation and purge (DEC-12)
`VAPID_PRIVATE_KEY`, `VAPID_PUBLIC_KEY`	Server / public key to browser	Private key server only
`SMS_API_KEY`	Server only	Never logged

Performance, reliability and compatibility targets: as NFR-01 to NFR-09. Hosting limits must be verified before the deployment phase (A-04).

7. System Architecture

QueuePulse is a modular monolith: one Next.js app on Vercel talks to one Supabase project. The database owns queue correctness; the app owns screens and notifications.

```
flowchart LR
  V["Visitor browser: live page"] -- "WebSocket subscribe" --> RT["Supabase Realtime"]
  V -- "HTTPS: token view function" --> DB
  S["Staff browser: dashboard"] -- "HTTPS" --> API["Next.js route handlers on Vercel"]
  AD["Admin browser"] -- "HTTPS" --> API
  API -- "user session: SQL and functions" --> DB[("PostgreSQL with RLS")]
  S -. "login" .-> AU["Supabase Auth"]
  AU -.-> DB
  DB -- "state broadcast on topic queue" --> RT
  API -- "Web Push" --> PS["Browser push services"]
  API -- "HTTPS" --> SMS["SMS gateway"]
```

Components

Component	Responsibility	Depends on
Public live page	Subscribe to one queue's state; compute people ahead and ETA for the visitor's token	Realtime, token view function
Staff dashboard	Issue token, Next, Skip, cancel, view reports	Route handlers
Route handlers	Validate input, call database functions, run notification outbox, serve reports	Database, Web Push, SMS
Postgres functions	`issue_token`, `advance_queue`; lock the queue row, update tokens and `queue_state` in one transaction	Tables
Notification outbox	Rows for alerts to send; marked sent or retried	Route handlers

Main flows

1. *Issue token:* Staff click Issue → handler validates → `issue_token` locks the queue row, takes the next number, inserts the token and updates `queue_state` → QR link is returned.
2. *Advance:* Staff click Next → `advance_queue` completes the current token, calls the next one, records the service duration, recalculates the average, updates `queue_state`, and writes outbox rows for tokens now 3 or fewer ahead → Realtime pushes the change → handler sends the pending alerts.
3. *Visitor view:* Page loads token through the token view function → subscribes to `queue_state` → shows ETA; polls every 10 seconds if the socket fails.

Trade-offs

Choice	Gain	Cost
Logic in Postgres functions	Correct under concurrent clicks	SQL logic must be tested in the database, not only in Jest
Public `queue`-state broadcast	No personal data over realtime	ETA for a token is computed in the browser from public numbers

Choice	Gain	Cost
Outbox for alerts	No lost alerts after a crash	One extra table and a retry step
Single deploy unit	Simple for 3 students	One failure takes down staff and visitor pages together

8. Blueprint Progress

Part	Contents	Status
1	Understanding, MVP, risks, technology, PRD, TRD, architecture	Defined (this document)
2	Detailed design, security and privacy, feature specifications, application flows, UI/UX	Defined (Sections 9–13)
3	Database schema and ER diagram, API contracts, project structure	Defined (Sections 14–16)
4	Implementation phases and the numbered AI coding prompt pack	Defined (Sections 17–18)
5	SQA plan and test cases, deployment, risk register, traceability and readiness review	Defined (Sections 19–22)

Nothing is implemented yet. All five parts are defined. Next: save this blueprint as docs/blueprint.md and run Prompt 1.

9. Detailed System Design

Part 1 fixed the structure. This section fixes how each module behaves.

Modules and data ownership

Module	Responsibility	Owns
Access	Resolve session, role, organization; block deactivated users	`profiles`, `organizations`

Module	Responsibility	Owns
Queue setup	Queue name, default minutes per patient, active flag	`queues`
Token issuing	Number allocation, public code, optional phone	`tokens`
Queue advance	Token state machine: Next, Skip, Cancel, Recall	`tokens`
Average service rule	FR-11 average, written once in SQL	`queue_state.avg_service_minutes`
Live state	Public counters visitors subscribe to	`queue_state`
Notifications	Outbox, Web Push, SMS adapter	`notification_outbox`, `push_subscriptions`
Reports	Read-only aggregates over tokens	nothing (derived)

Rules: clients never write tables directly; only Postgres functions change `tokens` and `queue_state`. The average rule lives in SQL only. The browser multiplies people ahead by the stored average and rounds; it contains no other business rule.

Token state machine

From	Action	To	Who
`waiting`	Next or Skip (lowest number)	`serving`	Staff
`serving`	Next	`completed`	Staff
`serving`	Skip	`skipped`	Staff
`waiting`	Cancel	`cancelled`	Staff
`skipped`	Recall (Should)	`waiting`, becomes next to be called	Staff

Critical workflow: `advance_queue(queue_id, action, expected_serving_token_id)`

1. Lock the queue row (`FOR UPDATE`); confirm the caller belongs to the queue's organization, the organization and queue are active, and the queue is not paused.

2. If `expected_serving_token_id` differs from the actual serving token, return a conflict and change nothing. This stops double-clicks and two staff advancing together.
3. Finish the serving token: `completed` (Next) with `service_seconds` = now minus `called_at`, or `skipped` (Skip) with no duration.
4. Call the lowest-numbered `waiting` token: set `serving` and `called_at`. If none remain, the queue becomes idle.
5. Recompute the average: mean `service_seconds` of the last 5 `completed` tokens from 20 to 3600 seconds (D-04); fewer than 5 uses those available; none uses the queue default.
6. Update `queue_state` (current number, last issued, average, status, `updated_at`).
7. Insert one outbox row for every token whose people ahead is now 3 or fewer and that has none yet (unique key on token, channel, kind).

People ahead = the `serving` token (if its number is lower) plus `waiting` tokens with a lower number. This gives the Part 1 example: serving #12, mine #18 → 6 ahead → 6 × 7 = 42 minutes.

Other workflows

- `issue_token(queue_id, phone, request_id)`: lock queue row, take last number + 1 for today's Asia/Dhaka date, insert token with random public code. `request_id` is unique per queue, so a retry after a network error returns the same token instead of creating a second one.
- Pause: store `paused_at`. Resume: shift the serving token's `called_at` forward by the paused time, so a pause never inflates service time.
- Cancel: waiting token becomes `cancelled`; it leaves the people-ahead count at once.

Transactions and failure handling

- Each function is one transaction. Sending push or SMS happens after commit, outside the transaction.
- Outbox rows are processed after each advance (up to 20 per run, 3 attempts). A row that keeps failing is marked `failed` and shown to staff; unsent rows are retried on the next staff action. This is a known MVP limit: with no further action, a failed alert is not retried.
- Staff screen on network error: shows "Not saved", keeps the button state, offers Retry; the server's conflict check makes Retry safe.
- Visitor page on lost connection: shows "Reconnecting", polls every 10 seconds, resumes realtime when possible.

Scalability: one queue action causes one `queue_state` update, so realtime fan-out per action is a single row change. Indexes are specified with the schema in Part 3.

10. Security and Privacy Specification

Controls are based on this application's actual risks: a public page, staff-only write actions, and optional visitor phone numbers. A checklist does not guarantee security; each control below is paired with a test in Part 5.

ID	Risk	Impact	Required control
SEC-01	Weak or shared staff login	Queue manipulated	Supabase Auth; public sign-up disabled; minimum password length 10; generic login errors; cookie-based session (httpOnly) through the Supabase SSR helper.
SEC-02	Staff read or change another organization's data	Data breach	Row Level Security enabled on every table; policies check `profiles.org_id` and `profiles.is_active`. Role comes from `profiles`, never from user-editable metadata.
SEC-03	Anonymous access to tokens or phone numbers	Privacy breach	Anonymous users can only read `queue_state` and run the token-view function by public code. No anonymous access to `tokens`, `profiles`, outbox or subscriptions.
SEC-04	Guessable token links	Others watch or cancel a visitor's token	Public code is random, at least 128 bits, never the serial number. The live page is read-only.
SEC-05	Bad input	Corrupt data, errors	Zod validation in route handlers; database constraints (types, lengths, state values); phone normalised to Bangladeshi mobile format.

ID	Risk	Impact	Required control
SEC-06	SQL injection	Full compromise	Supabase client and RPC only; no SQL built from strings; every function sets a fixed `search_path`.
SEC-07	Secret leakage	Full compromise	Service-role, VAPID private and SMS keys only in server environment variables; `.env*` ignored by git; secret scan before release; different keys per environment.
SEC-08	Abuse of public endpoints	Cost, downtime	Platform and Supabase Auth rate limits; push-subscribe endpoint requires a valid public code. A dedicated limiter is **Partially Defined**: decide in Part 4.
SEC-09	CSRF and XSS	Unauthorized actions	`SameSite=Lax` cookies and Origin check on state-changing handlers; React output escaping; no `dangerouslySetInnerHTML`; Content-Security-Policy and standard security headers.
SEC-10	No trace of who acted	Disputes, no accountability	Each Next, Skip, Cancel and Recall stores the acting user and time.
SEC-11	Phone numbers kept too long	Privacy breach	Optional field with consent text; masked in staff screens; excluded from logs and realtime; deleted 7 days after the token day by a daily purge job (pg_cron or Vercel Cron; availability to be verified).
SEC-12	Verbose errors and logs	Information leak	Clients get generic messages with a request ID; server logs hold no phone numbers or keys.

ID	Risk	Impact	Required control
SEC-13	Vulnerable dependencies	Compromise	Lockfile, `npm audit` and Dependabot; audit result is a release gate.
SEC-14	Data loss	Lost reports	Migrations and seeds in git; Supabase backup features depend on plan (verify); weekly manual export for the demo.
SEC-15	Deactivated staff keep access	Misuse after leaving	RLS checks `is_active` on every request, so deactivation is immediate.

Residual risk (SEC-03), resolved in Section 14: realtime now uses Broadcast from Database on an unguessable topic per queue, so no table needs anonymous read. What remains: anyone who knows a queue's UUID (it appears in the QR link) can watch that queue's counters, which hold no personal data. The public-topic behavior must be confirmed against current Supabase documentation in the first database phase.

Hard rules: authorization is always enforced on the server and in the database; the service-role key never reaches the browser; the live page never receives phone numbers.

11. Feature Specifications

Acceptance criteria are in the PRD table (Section 5); each feature below adds behavior, inputs, permissions and error states.

Feature	Behavior and rules	Inputs and validation	Permissions	Edge cases and error states
F-01 Sign-in and reset (FR-01, 02)	Email and password; reset by emailed link	Valid email; password 10+ characters	Anyone	Wrong credentials: one generic error. Expired reset link: ask to request a new one. Deactivated user: sign-in refused.
F-02 Organization and staff admin (FR-03, 04, 16)	Admin creates organizations and staff, deactivates either, sees counts	Name 2–80 characters; unique staff email	`admin` only	Duplicate email shows a clear error. Deactivating an organization blocks its staff at once; queued tokens stay readable to visitors.
F-03 Queue setup (FR-05)	Create, rename, deactivate queue; set default minutes per patient	Name 2–60 characters; default minutes 1–120	`staff` of that organization	Deactivating a queue with waiting tokens needs confirmation.
F-04 Issue token and slip (FR-06, 07)	Next number for today; shows number, QR, link, print button	Optional phone: 11 digits starting 013–019, normalised; `request_id`	`staff` of that organization	Network error then retry returns the same token. Paused or inactive queue: issuing refused with reason.

Feature	Behavior and rules	Inputs and validation	Permissions	Edge cases and error states
F-05 Next, Skip, Cancel (FR-08, 09, 13)	Runs `advance_queue`; Cancel only for `waiting` tokens	Queue ID, action, expected serving token	`staff` of that organization	Conflict: screen refreshes and says "Someone already advanced the queue". No waiting token: queue shows idle. Next with nothing serving simply calls the first waiting token.
F-06 Visitor live view (FR-10, 11)	Shows serving number, own number, people ahead, ETA, status	Public code from the URL	Anyone with the link	Unknown code: "Token not found". Day over: "This token has expired". Skipped: "You were skipped, please contact the desk". Cancelled and completed shown plainly. ETA shown as "about N minutes".

Feature	Behavior and rules	Inputs and validation	Permissions	Edge cases and error states
F-07 Near-turn alert (FR-12)	Once per token when people ahead is 3 or fewer: in-page banner and vibration; Web Push if allowed	Push permission requested only after a tap on "Alert me"	Visitor, own token only	Permission denied: page explains and offers SMS if a phone was given. Alert is never repeated on refresh.
F-08 Staff dashboard (FR-14)	Current token, next 5 waiting, counts, current average, big Next and Skip buttons	None	`staff` of that organization	Buttons disabled while a request is running. Offline banner when the connection drops.
F-09 Reports (FR-15)	Tokens issued, completed, skipped; average wait (issued to called); average service	Queue, date range up to 31 days	`staff` of own organization; `admin` for all	Empty range: "No data for this period".
F-10 SMS alert (FR-17, Should)	Same trigger as F-07; one message per token	Provider adapter; mock in development	Server only	Provider failure: retried up to 3 times then `failed`, visible to staff.
F-11 Pause and recall (FR-18, Should)	Pause hides ETA and blocks Next; resume corrects service time; recall one skipped token	Queue ID, token ID	`staff` of that organization	Recall of a token not `skipped` is refused.
F-12 Peak hours and CSV (FR-19, Should)	Hourly counts; CSV of the report	Same as F-09	As F-09	Large range limited to 31 days.

12. Application Flow and Navigation

Routes and entry points

Route	Who	Purpose
`/login`, `/reset`	Staff, admin	Sign in and password reset
`/dashboard`	Staff	List of own queues
`/dashboard/queues/[id]`	Staff	Controller: issue token, Next, Skip, Cancel
`/dashboard/reports`	Staff	Reports
`/admin`	Admin	Organizations, staff, overview
`/t/[code]`	Visitor	Live token page (QR target)

After login, staff go to `/dashboard` and admin to `/admin`. Visitors only ever arrive through the QR link or a shared link.

Primary journey: from token to turn

```
flowchart TD
  A["Staff issues token"] --> B["Slip with QR shown or printed"]
  B --> C["Visitor scans QR"]
  C --> D["Live page: serving number, people ahead, ETA"]
  D --> E{"Staff presses Next or Skip"}
  E --> F["Screens update within 2 seconds"]
  F --> G{"3 or fewer ahead?"}
  G -- "No" --> D
  G -- "Yes" --> H["One alert: banner, push or SMS"]
  H --> I{"Token called"}
  I -- "Completed" --> J["Done"]
  I -- "Skipped" --> K["Message: contact the desk"]
  C -. "Connection lost" .-> L["Reconnecting and polling every 10 seconds"]
  L -.-> D
```

Failure and recovery paths

Situation	What happens
Staff network drops during Next	"Not saved" with Retry; conflict check prevents a double advance

Situation	What happens
Two staff press Next together	Second request gets a conflict, screen refreshes
Visitor opens an expired or wrong link	Clear message, no data shown
Push permission denied	Page explains; in-page alert still works; SMS if a phone was given
System is down	Staff continue manually (NFR-03); enter the current state when the system returns

Administrative workflow: admin creates an organization → creates a staff account for it → shares the first sign-in details securely → staff set their password through reset → staff create a queue and start issuing tokens.

13. UI/UX Design Specification

No mockups exist yet; this section defines what each screen must contain.

Design principles: one primary action per screen; large numbers readable from a distance; mobile first for visitors, tablet and desktop first for staff; plain wording in Bangla and English; state is always shown in words, not colour alone.

Screen inventory

Screen	Purpose	Essential elements
Login / Reset	Authenticate	Email, password, error text, reset link
Queue list	Choose a queue	Queue name, current serving number, waiting count, active flag
Controller	Run the queue	Serving number (very large), Next and Skip buttons, Issue token form, next 5 waiting, average minutes, Cancel per waiting token
Token slip	Hand the visitor a link	Queue name, token number, date, QR, short link, Print button
Reports	Review a period	Queue and date filters, totals, average wait and service, table

Screen	Purpose	Essential elements
Admin	Manage the platform	Organization list and form, staff list and form, today overview
Live page	Visitor follows the queue	Serving number, own number, people ahead, "about N minutes", status line, connection status, "Alert me" button

Visual rules

Item	Rule
Typography	System font stack for Latin; Noto Sans Bengali for Bangla; base 16 px; serving number 64 px bold on the live page
Spacing	4 px scale (4, 8, 12, 16, 24, 32); 16 px side gutter on mobile
Colour	Primary `#0F766E`, text `#0F172A`, background `#FFFFFF`, warning `#B45309`, danger `#B91C1C`, success `#15803D`; contrast to be verified with a checker in Part 5
Touch targets	At least 44 × 44 px

Reusable components (only these for the MVP): Button, TextField, Select, Table, StatusBadge, Banner, ConfirmDialog, BigNumber, QRCard.

Forms and interaction: labels above fields; validation on submit with the message next to the field; submit buttons disabled while a request runs; destructive actions (cancel token, deactivate) ask for confirmation.

Responsive behavior: live page and slip work from 360 px; controller stacks vertically on phone and uses two columns from 768 px; admin and reports tables scroll horizontally on small screens.

Accessibility: every input has a label; visible focus ring; buttons reachable by keyboard; status changes on the live page announced through an `aria-live` region; all text meets WCAG 2.1 AA contrast; the alert banner does not rely on sound alone.

States for every data screen

State	Behavior
Loading	Skeleton or spinner with text
Empty	Short sentence plus the next action (for example "Issue the first token")
Success	Brief confirmation near the action
Error	Plain message, Retry where safe
Disabled	Button greyed with the reason available as text
Confirmation	Dialog naming the action and the token or queue affected

14. Database Design and Schema

PostgreSQL on Supabase, 8 tables. Only database functions change tokens and live queue data; the browser and route handlers never write those tables directly.

Decisions made in this part

ID	Decision	Why
DEC-03 (revised)	Realtime uses Broadcast from Database: a trigger on `queue_state` sends the new counters to a public topic `queue:<queue_id>`. No anonymous read on any table.	Supabase documents Broadcast from Database and recommends Broadcast over Postgres Changes for scale. This removes the accepted residual risk in SEC-03. The exact `realtime.send` parameters for a public (non-private) topic must be confirmed against current Supabase documentation in the first database phase.

ID	Decision	Why
DEC-08	Queue-changing functions are `SECURITY DEFINER`, check `auth.uid()` and organization inside, set a fixed `search_path`, and have execute revoked from public. No INSERT or UPDATE policies exist on `tokens` or `queue_state`.	One enforced write path; row locks and rules cannot be bypassed.
DEC-09	`queue_state.waiting_numbers` holds the numbers of waiting tokens (no personal data).	The browser computes people ahead exactly, even when tokens are cancelled or skipped, without reading `tokens`.
DEC-10	New token status `expired` for tokens left over at day rollover.	Part 1 listed 5 states; F-06 already needs "expired".
DEC-11	Day rollover is lazy: the first issue or advance on a new Asia/Dhaka day resets the queue state; `get_token_view` reports `expired` by date even before that.	No scheduler needed for correctness.
DEC-12	The service-role client is created in one file and used by four server modules only: outbox processing, push registration, staff-account creation, purge.	Keeps the key's use auditable (SEC-07).

Entities

`organizations`

Column	Type	Rules
`id`	uuid	PK, default `gen_random_uuid()`
`name`	text	NOT NULL, 2–80 characters
`is_active`	boolean	NOT NULL, default true
`created_at`	timestamptz	NOT NULL, default `now()`

`profiles`

Column	Type	Rules
`id`	uuid	PK, FK `auth.users(id)` ON DELETE CASCADE
`role`	text	NOT NULL, `admin` or `staff`
`org_id`	uuid	NULL, FK `organizations(id)`; CHECK: staff must have it, admin must not
`full_name`	text	NOT NULL, 2–80 characters
`is_active`	boolean	NOT NULL, default true
`created_at`	timestamptz	NOT NULL, default `now()`

`queues`

Column	Type	Rules
`id`	uuid	PK, default `gen_random_uuid()`
`org_id`	uuid	NOT NULL, FK `organizations(id)`
`name`	text	NOT NULL, 2–60 characters; UNIQUE with `org_id`
`default_service_minutes`	smallint	NOT NULL, default 5, 1–120
`is_active`	boolean	NOT NULL, default true
`created_at`	timestamptz	NOT NULL, default `now()`

`queue_state` (one row per queue; public data only)

Column	Type	Rules
`queue_id`	uuid	PK, FK `queues(id)`
`service_date`	date	NOT NULL, Asia/Dhaka date this state belongs to
`serving_number`	integer	NULL when idle
`last_issued`	integer	NOT NULL, default 0

Column	Type	Rules
`waiting_numbers`	integer[]	NOT NULL, default empty
`avg_service_minutes`	numeric(5,1)	NOT NULL, from the FR-11 rule
`status`	text	NOT NULL, `active` or `paused`
`paused_at`	timestamptz	NULL
`version`	bigint	NOT NULL, increases on every change; clients ignore older messages
`updated_at`	timestamptz	NOT NULL, default `now()`

`tokens`

Column	Type	Rules
`id`	uuid	PK, default `gen_random_uuid()`
`queue_id`	uuid	NOT NULL, FK `queues(id)`
`service_date`	date	NOT NULL
`number`	integer	NOT NULL, 1 or more; UNIQUE (`queue_id` , `service_date` , `number`)
`public_code`	text	NOT NULL, UNIQUE, 22 characters from 128 random bits
`status`	text	NOT NULL, default `waiting` ; one of `waiting` , `serving` , `completed` , `skipped` , `cancelled` , `expired`
`phone`	text	NULL; CHECK `^\+8801[3-9][0-9]{8}$` (stored in international format)
`request_id`	uuid	NOT NULL; UNIQUE (`queue_id` , `request_id`)
`issued_at`	timestamptz	NOT NULL, default `now()`
`called_at`	timestamptz	NULL; required when `serving`
`finished_at`	timestamptz	NULL
`service_seconds`	integer	NULL; required when `completed`

`push_subscriptions`

Column	Type	Rules
`id`	uuid	PK
`token_id`	uuid	NOT NULL, FK `tokens(id)` ON DELETE CASCADE
`endpoint`	text	NOT NULL, UNIQUE
`p256dh` , `auth`	text	NOT NULL
`created_at`	timestamptz	NOT NULL, default `now()`

`notification_outbox`

Column	Type	Rules
`id`	bigint	PK, identity
`token_id`	uuid	NOT NULL, FK `tokens(id)` ON DELETE CASCADE
`channel`	text	NOT NULL, `push` or `sms`
`kind`	text	NOT NULL, `near_turn`
`status`	text	NOT NULL, `pending`, `sent` or `failed`; default `pending`
`attempts`	smallint	NOT NULL, default 0
`last_error`	text	NULL, never contains the phone number
`created_at`, `sent_at`	timestamptz	`created_at` default `now()`; `sent_at` NULL
		UNIQUE (`token_id`, `channel`, `kind`) gives "once per token"

`audit_events` (SEC-10)

Column	Type	Rules
`id`	bigint	PK, identity
`queue_id`	uuid	NOT NULL, FK `queues(id)`
`token_id`	uuid	NULL, FK `tokens(id)`
`actor_id`	uuid	NOT NULL, FK `profiles(id)`
`action`	text	NOT NULL, `issue`, `next`, `skip`, `cancel`, `recall`, `pause`, `resume`
`created_at`	timestamptz	NOT NULL, default `now()`

ER diagram

```
erDiagram
  AUTH_USERS ||--|| PROFILES : "has"
  ORGANIZATIONS ||--o{ PROFILES : "employs"
  ORGANIZATIONS ||--o{ QUEUES : "owns"
  QUEUES ||--|| QUEUE_STATE : "has live state"
  QUEUES ||--o{ TOKENS : "issues"
  TOKENS ||--o{ PUSH_SUBSCRIPTIONS : "has"
  TOKENS ||--o{ NOTIFICATION_OUTBOX : "triggers"
  PROFILES ||--o{ AUDIT_EVENTS : "performs"
  QUEUES ||--o{ AUDIT_EVENTS : "logs"
```

Indexes (each tied to a query)

Index	Query it serves
`tokens (queue_id, service_date, status, number)`	Find lowest waiting token; dashboard next 5; counts
`tokens (public_code)` unique	`get_token_view`
`tokens (queue_id, finished_at DESC) WHERE status = 'completed'`	Last 5 completed for the average
`tokens (service_date)`	Daily purge, reports
`notification_outbox (created_at) WHERE status = 'pending'`	Outbox processing
`profiles (org_id)`, `queues (org_id)`	Organization lookups in RLS and lists
`audit_events (queue_id, created_at)`	Review of actions

Row Level Security (enabled on every table)

Table	Anonymous	Staff	Admin
`organizations`	none	read own	read, create, update
`profiles`	none	read own row	read, create (through server), update

Table	Anonymous	Staff	Admin
`queues`	none	read, create, update in own organization	read
`queue_state`	none	read in own organization	read
`tokens`	none	read in own organization (phone masked by the server)	none; reports through functions
`push_subscriptions`, `notification_outbox`	none	none	none; server only
`audit_events`	none	none	read

Policies use helper functions `current_org_id()` and `is_admin()`, which return a value only for an active profile. Visitors reach data only through `get_token_view` and the broadcast topic.

Migration and seed strategy: Supabase CLI migrations in git, one reviewed migration per change; `seed.sql` with 1 admin, 2 organizations, 3 queues and sample tokens for local use; pgTAP tests in `supabase/tests` for RLS, numbering, advance and the average rule.

Retention and integrity: `purge_expired_pii()` clears `phone` and deletes push subscriptions and outbox rows for tokens older than 7 days; token rows stay (without personal data) for reports; audit events are kept 180 days. Organizations and queues are deactivated, never deleted (foreign keys RESTRICT).

15. Backend Logic and API Contracts

Two layers: Postgres functions hold the rules; Next.js route handlers authenticate, validate with Zod, call the functions and shape the response. Only MVP endpoints are listed.

Conventions

- JSON in and out. Errors use one shape: `{ "error": { "code", "message", "requestId" } }`.
- A resource that belongs to another organization returns 404, the same as a missing one.

- Validation failures return 400 with field messages. No stack traces or SQL errors reach the client (SEC-12).

Error code	HTTP	Meaning
`VALIDATION_FAILED`	400	Bad or missing input
`UNAUTHENTICATED`	401	No valid session
`FORBIDDEN`	403	Wrong role or deactivated account or organization
`NOT_FOUND`	404	Missing, or not visible to the caller
`STALE_STATE`	409	`expected_serving_token_id` no longer matches; response includes the current state
`QUEUE_PAUSED`, `QUEUE_INACTIVE`	409	Action not allowed in this queue state
`TOKEN_NOT_WAITING`, `NOTHING_SERVING`	409	Cancel or Skip on a token or queue in the wrong state; recall adds TOKEN_NOT_SKIPPED and ALREADY_RECALLED (DEC-17)
`DUPLICATE_NAME`	409	Queue or organization name already used
`RATE_LIMITED`	429	Too many requests
`INTERNAL`	500	Unexpected failure, logged with `requestId`

Database function contracts

Function	Callable by	Input	Output	Rules and errors
`get_token_view(p_code)`	anonymous, staff	Public code	One row: token number, status, queue ID, queue name, service date, serving number, waiting numbers, average minutes, queue status, version. No row if unknown.	Returns `expired` when the token's date is before today (Asia/Dhaka). Never returns phone, token ID or organization.
`issue_token(p_queue_id, p_phone, p_request_id)`	active staff of the queue's organization	Queue, optional phone, request ID	Token number, public code, status	Rolls the day if needed; refuses paused or inactive queue; same `request_id` returns the same token.
`advance_queue(p_queue_id, p_action, p_expected_serving_token_id)`	active staff of the organization	`next` or `skip`, expected serving token or null	New queue state	Steps in Section 9; `STALE_STATE`, `QUEUE_PAUSED`, `NOTHING_SERVING` (Skip with nobody serving).
`cancel_token(p_token_id)`	active staff of the organization	Token ID	New queue state	Only `waiting` tokens; writes audit event.
`pause_queue`, `resume_queue`, `recall_token` (Should)	active staff of the organization	Queue or token ID	New queue state	Recall only `skipped` tokens, once.

Function	Callable by	Input	Output	Rules and errors
`report_summary(p_queue_id, p_from, p_to)`	staff of the organization, admin	Queue (or all for admin), dates, range up to 31 days	Counts by status, average wait seconds, average service seconds, hourly counts	Aggregates only; no phone numbers.
`enqueue_near_turn(p_queue_id)`	internal only	Queue	none	Creates missing outbox rows for tokens with 3 or fewer ahead that have a phone or push subscription. Called by issue, advance, cancel, recall and push registration.
`register_push(p_code, p_endpoint, p_p256dh, p_auth)`	server (service role) only	Public code, subscription	none	Token must be `waiting` or `serving`; then calls `enqueue_near_turn`.
`purge_expired_pii()`	scheduler or server only	none	Rows cleaned	Section 14, retention.

A trigger on `queue_state` sends the new state to topic `queue:<queue_id>` after every change (DEC-03).

Route handlers

Endpoint	Auth	Request	Success	Errors
`POST /api/public/tokens/[code]/push`	none	`{ subscription: { endpoint, keys: { p256dh, auth } } }`	204	400, 404, 409 `TOKEN_NOT_WAITING`, 429
`POST /api/queues`	staff	`{ name, default_service_minutes }`	201 queue	400, 401, 403, 409 `DUPLICATE_NAME`
`PATCH /api/queues/[id]`	staff	Any of `name`, `default_service_minutes`, `is_active`	200 queue	400, 401, 403, 404, 409

Endpoint	Auth	Request	Success	Errors
`POST /api/queues/[id]/tokens`	staff	`{ phone?, request_id }`	201 `{ token: { id, number, public_code, status }, link }`; 200 when `request_id` repeats	400, 401, 403, 404, 409 `QUEUE_PAUSED` or `QUEUE_INACTIVE`
`POST /api/queues/[id]/advance`	staff	`{ action: "next" or "skip", expected_serving_token_id }`	200 `{ state }`, then pending alerts are sent	400, 401, 403, 404, 409 `STALE_STATE`, `QUEUE_PAUSED`, `NOTHING_SERVING`
`POST /api/queues/[id]/tokens/[tokenId]/cancel`	staff	none	200 `{ state }`	401, 403, 404, 409 `TOKEN_NOT_WAITING`
`GET /api/queues/[id]/dashboard`	staff	none	200 `{ state, serving, next_waiting (5, phone masked), counts }`	401, 403, 404
`GET /api/reports`	staff, admin	Query: `queue_id`, `from`, `to`	200 summary	400 (range over 31 days), 401, 403, 404
`POST /api/queues/[id]/pause`, `/resume`; `POST /api/queues/[id]/tokens/[tokenId]/recall` (Should)	staff	none	200 `{ state }`	401, 403, 404, 409
`GET /api/reports/export` (Should)	staff, admin	As reports	CSV	As reports
`GET, POST /api/admin/organizations`; `PATCH /api/admin/organizations/[id]`	admin	Name, `is_active`	200 or 201 organization	400, 401, 403, 404, 409
`GET, POST /api/admin/staff`; `PATCH /api/admin/staff/[id]`	admin	Email, full name, organization, `is_active`	200 or 201 staff (creation uses the Auth admin API through the service-role client)	400, 401, 403, 404, 409
`GET /api/admin/overview`	admin	none	200 `{ organizations, active_queues, tokens_today }`	401, 403

The visitor page does not need a public route for reading: it calls `get_token_view` with the public API key and subscribes to the broadcast topic. If the broadcast connection fails it calls `get_token_view` every 10 seconds.

Service responsibilities

- *Advance handler:* validate → call `advance_queue` → after commit, process up to 20 pending outbox rows (push through the web-push library, SMS through the adapter) → mark `sent` or increment `attempts`, `failed` after 3 → return the state. A failure while sending never rolls back the queue change.
- *Push registration handler:* validate with Zod → check rate limit → call `register_push` with the service-role client.
- *Business rules are not duplicated:* numbering, state changes, the average and the near-turn rule live in SQL; the browser only computes people ahead from `waiting_numbers` and multiplies by the average.

External services: Web Push uses the standard Push API with VAPID keys, no third-party account. The SMS gateway is not chosen (D-02); its documentation, authentication, price and failure behavior must be verified before FR-17 is built. The adapter interface is `send(to, text) → { ok, error? }`.

16. Standard Project Structure

Feature folders for product code, shared folders for reusable parts, and a separate `supabase` folder for everything that lives in the database. Small enough for a 3-person team to hold in their heads.

```
queuepulse/
├─ src/
|  ├─ app/                      Next.js App Router
|  |  ├─ (public)/t/[code]/     Visitor live page
|  |  ├─ (staff)/login/ dashboard/ reports/
|  |  ├─ (admin)/admin/
|  |  └─ api/                   Route handlers (Section 15)
|  ├─ features/
|  |  ├─ queue/                 Controller UI, token slip, schemas, eta.ts
|  |  ├─ live/                  Realtime subscription, polling fallback
|  |  ├─ notifications/         Push client, outbox processing, SMS adapter
|  |  ├─ reports/
|  |  └─ admin/
|  ├─ components/ui/            Button, TextField, Table, Banner, ...
|  ├─ lib/
|  |  ├─ supabase/              browser.ts, server.ts, service.ts
|  |  ├─ http/                  Error shape, handler wrapper, rate limit
|  |  ├─ time.ts                Asia/Dhaka date helpers
|  |  └─ env.ts                 Validated environment variables
```

```
|  ├─ i18n/                       bn.ts, en.ts
|  └─ middleware.ts               Session refresh, route protection
├─ public/sw.js                   Service worker for Web Push
├─ supabase/
|  ├─ migrations/                 One reviewed SQL file per change
|  ├─ tests/                      pgTAP: RLS, numbering, advance, average
|  ├─ seed.sql
|  └─ config.toml
├─ tests/
|  ├─ unit/                       Jest
|  ├─ e2e/                        Playwright
|  └─ api/                        Postman collection
├─ docs/                          This blueprint, decision log
├─ .env.example
└─ package.json, tsconfig.json, eslint config
```

Path	Why it exists
`features/queue/eta.ts`	The only TypeScript business rule: `peopleAhead(waitingNumbers, servingNumber, myNumber)` and `etaMinutes(peopleAhead, avgMinutes)`; unit-tested with Jest.
`lib/supabase/service.ts`	The only place the service-role client is created (DEC-12). Importing it from a client component must fail the lint rule or build.
`lib/http/`	One wrapper gives every handler auth check, Zod parsing, error shape and request ID, so handlers stay short.
`supabase/tests/`	The average rule and numbering live in SQL, so they are tested in the database, not in Jest.
`lib/env.ts`	Fails at start-up when a required secret is missing instead of failing later at runtime.
`i18n/`	Plain dictionaries for Bangla and English (D-01); no i18n library for the MVP.

No separate `services/` or `repositories/` layers: the database functions are the service layer, and handlers call them directly.

17. Development and Implementation Plan

Ten phases, each small enough for one AI coding session and one review. A phase starts only when the previous one meets its acceptance criteria. Suggested owners follow the roles in the proposal: **DB** (database), **FE/PM** (frontend and project management), **BE** (backend and API).

Phase	Objective	Depends on	Delivers	Owner	Week	Status
1	Foundation and tooling	none	Repo, env validation, error wrapper, CI	FE/PM	2	Not started
2	Schema, RLS, realtime spike	1	8 tables, policies, seed, broadcast trigger, pgTAP	DB	3	Not started
3	Queue functions in SQL	2	Issue, advance, cancel, average rule, near-turn rule, token view (FR-06, 08, 09, 11, 12, 13)	DB	3–4	Not started
4	Auth, access control, admin	1, 2	Login, reset, roles, admin pages (FR-01–04, 16; SEC-01, 02, 15)	BE	4–5	Not started
5	Staff controller	3, 4	Queues, tokens, slip with QR, Next/Skip/Cancel (FR-05–09, 13, 14)	BE + FE/PM	5–6	Not started
6	Visitor live page and realtime	3, 5	Live page, ETA, fallback, Bangla/English, in-page alert (FR-10–12)	FE/PM	6–7	Not started
7	Notifications and purge	3, 6	Web Push, outbox, SMS adapter (mock), PII purge (FR-12, 17; SEC-11)	BE	7	Not started
8	Reports and Should features	3, 5	Reports (FR-15); pause, recall, peak hours, CSV (FR-18, 19)	BE + DB	7–8	Not started
9	Security hardening and regression	1–8	CSP, security matrix, accessibility, load and speed evidence	All	8–9	Not started
10	Deployment and release	9	Runbook, production setup, smoke test, rollback	All	10	Not started

Each phase's tasks, tests and acceptance criteria are in its prompt (Section 18). The requirement-to-phase-to-test matrix is completed in Part 5.

Decisions made in this part

ID	Decision	Why
DEC-13	Push-subscribe rate limit is best effort: in-memory per server instance, plus platform limits. Stated as a limitation, not a guarantee.	SEC-08 was Partially Defined; a database-backed limiter adds a table for little gain at this scale.
DEC-14	Admin creates staff with a random temporary password shown once to the admin; staff then use password reset.	No public sign-up (C-01); no email invitations to configure.
DEC-15	Staff screen uses action responses, focus refresh and 10-second polling in Phase 5, then switches to realtime in Phase 6.	Lets Phase 5 be tested without the realtime hook.
DEC-16	Purge scheduling (pg_cron or Vercel Cron) is chosen in Phase 7 after checking current plan limits.	Availability is unverified today (A-04).
DEC-17	Two error codes are added to Section 15: `TOKEN_NOT_SKIPPED` and `ALREADY_RECALLED` (409), used by recall.	Recall rules need their own messages.

18. AI Coding Prompt Pack

How to use it

1. Export this blueprint as `docs/blueprint.md` in the repository, so the coding tool can read the sections the prompts name.
2. For each phase, paste the **Shared Block** first, then that phase's prompt, into the coding tool. Run one phase at a time.
3. Read the handoff. Continue only when the acceptance criteria are met or a blocker is written down. Commit after each phase.
4. Deliberate deviation from "each prompt self-contained": the Shared Block (context, repository rules, quality rules, handoff format) is written once instead of repeated ten times. Pasted together, each prompt is self-contained. Ask for fully inlined files if you prefer.

Shared Block (paste before every phase prompt)

```
ROLE
You are a senior full-stack engineer implementing ONE phase of the QueuePulse
MVP. Work only inside this phase's scope. The full blueprint is in
docs/blueprint.md; the phase prompt names the sections to read first. If the
blueprint and the prompt conflict, stop and report the conflict instead of
choosing.

PRODUCT
QueuePulse is a real-time queue and wait-time web app for doctor's chambers,
diagnostic centers, banks and service centers in Bangladesh. Staff issue
tokens and press Next or Skip. Visitors open a QR link (no login, no app) and
see the serving number, people ahead and an estimated wait.

STACK
Next.js (App Router) with TypeScript strict and Tailwind CSS; Supabase
(PostgreSQL, Auth, Realtime Broadcast from Database, Row Level Security);
Zod; Jest (unit), pgTAP (database), Playwright (end-to-end), Postman (API);
Vercel and Supabase cloud.

ARCHITECTURE RULES (already decided; do not change)
1. Modular monolith. No microservices, message queues or extra
infrastructure.
2. Only database functions change tokens and queue_state. They are SECURITY
DEFINER, check auth.uid() and the organization inside, set a fixed
search_path, and have EXECUTE revoked from PUBLIC. No INSERT or UPDATE
policies exist on those tables.
3. Each business rule lives once: numbering, state changes, the average-
service rule and the near-turn rule are in SQL. The browser only computes
people ahead from waiting_numbers and multiplies by the stored average
(src/features/queue/eta.ts).
4. People ahead = the serving token (if its number is lower than mine) +
waiting tokens with a lower number. ETA = people ahead x avg_service_minutes.
5. Roles: admin (platform) and staff (one organization). Visitors are
anonymous and reach data only through get_token_view and the public broadcast
topic queue:<queue_id>. No anonymous table access.
6. The service-role Supabase client is created only in
src/lib/supabase/service.ts and imported only by: outbox processing, push
registration, staff-account creation, purge. Never in client code.
7. Token statuses: waiting, serving, completed, skipped, cancelled, expired.
The queue day is the Asia/Dhaka date.
8. API errors use { error: { code, message, requestId } }. A resource of
another organization returns 404.
9. Phone numbers are optional, stored as +8801XXXXXXXXX, never logged, never
```

```
sent to the live page, masked in staff screens.

REPOSITORY INSTRUCTIONS
Inspect existing files before changing anything. Follow current conventions.
Reuse working code. Do not overwrite unrelated work. Before adding a package,
check whether the platform or existing dependencies already cover the need,
and say why the package is required. Do not regenerate the whole project.

QUALITY REQUIREMENTS
TypeScript strict; no `any` without a comment. Every screen handles loading,
empty, success, error and disabled states where relevant. Validate input with
Zod in handlers and enforce permissions on the server. No hardcoded secrets;
read configuration through src/lib/env.ts. No unrelated scope expansion and
no silent deletion of working features. Update the tests and docs your change
affects.

VERIFICATION AND HANDOFF
Run the checks listed in the phase and report their real output. Your final
message must contain: what was implemented; files created and modified;
commands run with actual results; failures, skipped checks and blockers;
deviations from the blueprint and their impact; prerequisites for the next
phase. Never claim success when a check failed or was not run.
```

Prompt 1 – Foundation and tooling

```
PHASE 1 - FOUNDATION AND TOOLING
Read first: blueprint Sections 4, 6, 15 (conventions and error codes), 16.

CURRENT STATE
New or near-empty repository. Inspect it first; if code already exists, adapt
to it instead of overwriting.

OBJECTIVE (in scope)
- Next.js (App Router, TypeScript strict), Tailwind CSS, ESLint, Prettier,
Jest, Playwright (installed and configured; one smoke test only).
- Folders from Section 16, only those needed now: src/app, src/features,
src/components/ui, src/lib/supabase, src/lib/http, src/i18n, supabase/,
tests/unit, tests/e2e, tests/api, docs/.
- src/lib/env.ts: Zod-validated environment. Public:
NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY,
NEXT_PUBLIC_VAPID_PUBLIC_KEY. Server only: SUPABASE_SERVICE_ROLE_KEY,
VAPID_PRIVATE_KEY, VAPID_SUBJECT, SMS_PROVIDER (default "mock"), CRON_SECRET.
Server variables must never reach a client bundle. Fail at start-up with a
```

```
message that lists every missing variable.
- src/lib/time.ts: todayInDhaka(now?: Date) returns YYYY-MM-DD for
Asia/Dhaka.
- src/lib/http: AppError(code, status, message) with the error codes of
Section 15; a handler wrapper that adds a requestId (crypto.randomUUID), maps
AppError, maps ZodError to VALIDATION_FAILED (400), maps anything else to
INTERNAL (500, logged with the requestId, no details to the client).
Authentication hooks come in Phase 4.
- GET /api/health returns { status: "ok" }.
- src/i18n/bn.ts and en.ts with a small shared key set and a typed t()
helper.
- next.config security headers: X-Content-Type-Options, Referrer-Policy, X-
Frame-Options DENY, Permissions-Policy. (CSP comes in Phase 9.)
- supabase init (configuration only), .env.example, .gitignore that covers
.env*, README with local setup steps, and a GitHub Actions workflow that runs
lint, typecheck, test and build.
- package.json scripts: dev, build, lint, typecheck, test, test:e2e, test:db
(supabase test db).

OUT OF SCOPE
Database tables, authentication, any product screen, Supabase SDK use beyond
placeholder files.

VERIFICATION AND ACCEPTANCE
Tests to write: unit tests for todayInDhaka (for example 2026-10-19T19:00:00Z
must give 2026-10-20), env validation (missing and valid), the handler
wrapper (AppError, ZodError and unknown-error shapes), the health route; one
Playwright smoke test that the home page loads.
Run: npm run lint, npm run typecheck, npm test, npm run build.
Acceptance: all four pass; /api/health works; the app fails fast with a clear
message when a required variable is missing; no server variable value appears
in the client build output (check with a sentinel value).
```

Prompt 2 – Database schema, RLS and realtime spike

```
PHASE 2 - DATABASE SCHEMA, RLS AND REALTIME SPIKE
Read first: blueprint Sections 5 (FR/NFR), 10 (SEC-02, SEC-03), 14.

CURRENT STATE
Phase 1 is complete (repository, tooling, env validation, handler wrapper).
No tables exist.

OBJECTIVE (in scope)
```

```
1. Migrations (one file per logical change) creating the 8 tables exactly as
in Section 14: organizations, profiles, queues, queue_state, tokens,
push_subscriptions, notification_outbox, audit_events. Include every column,
type, NOT NULL, default, CHECK (profiles role/org rule; phone regex
^\+8801[3-9][0-9]{8}$; token status set including expired; called_at required
when serving; service_seconds required when completed; name lengths;
default_service_minutes 1-120), UNIQUE constraint and foreign key (RESTRICT,
except the cascades listed).
2. Every index listed in Section 14.
3. Helper functions current_org_id() and is_admin(): SECURITY DEFINER,
STABLE, fixed search_path. current_org_id() returns a value only for an
active staff profile of an active organization; is_admin() only for an active
admin profile.
4. RLS enabled on every table with the policies of the RLS table in Section
14. No INSERT, UPDATE or DELETE policies on tokens, queue_state,
push_subscriptions, notification_outbox or audit_events. Revoke all table
privileges from anon.
5. A trigger on queue_state (AFTER INSERT OR UPDATE) that broadcasts {
service_date, serving_number, last_issued, waiting_numbers,
avg_service_minutes, status, version } to the topic 'queue:' || queue_id.
6. REALTIME SPIKE (do this first). Check the CURRENT Supabase Realtime
documentation for realtime.send and realtime.broadcast_changes, and whether
an anonymous client can receive messages on a public (non-private) topic
without Realtime Authorization. Record the findings with documentation links
in docs/decisions.md. Then prove it with scripts/realtime-spike.ts: a client
using only the anon key subscribes to the topic, a row in queue_state is
updated, and the message arrives. If an anonymous client cannot receive
public-topic broadcasts, STOP and report the options (for example anonymous
sign-in with private-channel policies). Do not choose silently.
7. supabase/seed.sql: 2 organizations, 3 queues (2 in the first organization,
1 in the second) with their queue_state rows. scripts/seed-users.ts (local
development only, uses the service role): 1 admin and 2 staff (one per
organization) through the Auth admin API, plus their profiles.

OUT OF SCOPE
Queue functions (Phase 3), API handlers, UI.

VERIFICATION AND ACCEPTANCE
pgTAP tests in supabase/tests: anon cannot select or write any table; staff
of organization A cannot see queues, queue_state or tokens of organization B;
inactive staff, and staff of an inactive organization, see nothing; invalid
phone, invalid status, staff profile without organization, duplicate queue
name in one organization, duplicate (queue, date, number) and duplicate
```

```
request_id are rejected; an authenticated user cannot INSERT into tokens or
queue_state directly.
Run: supabase db reset, npm run test:db, npm run lint, npm run typecheck.
Acceptance: migrations and seed apply cleanly; all pgTAP tests pass; the
spike result is documented; the schema matches Section 14 (list any
deviation).
```

Prompt 3 – Queue functions (core rules in SQL)

```
PHASE 3 - QUEUE FUNCTIONS (CORE BUSINESS RULES IN SQL)
Read first: blueprint Sections 5 (FR-06, 08, 09, 11, 12, 13), 9, 14, 15
(database function contracts).

CURRENT STATE
Phase 2 is complete: schema, RLS, broadcast trigger, seed data, pgTAP setup.

OBJECTIVE (in scope)
All functions are SECURITY DEFINER with a fixed search_path and EXECUTE
revoked from PUBLIC. Each caller-facing function checks auth.uid(): active
profile, active organization, queue in the caller's organization. Failures
raise an exception whose message is exactly the API error code (for example
'STALE_STATE', 'FORBIDDEN', 'NOT_FOUND').
1. roll_queue_day(queue_id), internal: if queue_state.service_date is not
today (Asia/Dhaka), mark leftover waiting and serving tokens of the old date
as expired, set serving_number null, last_issued 0, waiting_numbers empty,
avg_service_minutes to the queue default, status active, paused_at null,
service_date today, and increase version.
2. issue_token(queue_id, phone, request_id), granted to authenticated: lock
the queue_state row FOR UPDATE; roll the day; refuse an inactive or paused
queue (QUEUE_INACTIVE, QUEUE_PAUSED); if (queue_id, request_id) already
exists return that token; otherwise number = last_issued + 1; public_code =
22 characters of base64url from 16 random bytes
(extensions.gen_random_bytes); insert the waiting token; append its number to
waiting_numbers; update last_issued; increase version; write an audit event
'issue'; call enqueue_near_turn. Return number, public_code, status.
3. advance_queue(queue_id, action, expected_serving_token_id), granted to
authenticated: implement steps 1-7 of Section 9 exactly. Lock; roll the day;
if the expected token differs from the real serving token raise STALE_STATE
and change nothing; Skip with nobody serving raises NOTHING_SERVING; paused
raises QUEUE_PAUSED. Next finishes the serving token as completed with
service_seconds = now minus called_at; Skip marks it skipped. Then call the
lowest-numbered waiting token (set serving and called_at) and remove its
number from waiting_numbers. Recompute avg_service_minutes: the mean
```

service_seconds of the last 5 completed tokens whose service_seconds is between 20 and 3600 (D-04); fewer than 5 uses those available; none uses queues.default_service_minutes; store with one decimal. Update queue_state, increase version, write the audit event, call enqueue_near_turn. If nothing is serving, Next simply calls the first waiting token. If nothing is waiting, the queue becomes idle. Return the queue_state row.
4. cancel_token(token_id), granted to authenticated: only waiting tokens (otherwise TOKEN_NOT_WAITING); status cancelled; remove from waiting_numbers; increase version; audit event; call enqueue_near_turn. Return the queue_state row.
5. enqueue_near_turn(queue_id), internal and NOT granted to any client role: for each waiting token whose people ahead is 3 or fewer (serving token counts when its number is lower), insert a pending notification_outbox row with channel 'sms' if the token has a phone and channel 'push' if it has a push subscription, kind 'near_turn', ON CONFLICT DO NOTHING.
6. get_token_view(code), granted to anon and authenticated: columns exactly as in Section 15; status is 'expired' when service_date is before today (Asia/Dhaka); never returns phone, token ID or organization; returns no row for an unknown code.

OUT OF SCOPE
Pause, resume, recall, reports, register_push, API handlers, UI.

VERIFICATION AND ACCEPTANCE
pgTAP tests: sequential numbering; day rollover (set queue_state.service_date to yesterday as the superuser, then call issue_token: leftovers become expired, numbering restarts at 1); request_id replay returns the same token; average rule with 0, 1, 3, 5 and 7 completed tokens, an outlier above 3600 seconds and a service under 20 seconds excluded, skipped tokens excluded; STALE_STATE changes nothing; NOTHING_SERVING; Next with nothing serving; Next with nothing waiting leaves the queue idle; cancel only for waiting tokens; near-turn rows: serving 12 with waiting 13-16, the row for 15 exists (3 ahead) and the row for 16 does not, and further advances never create a second row for the same token and channel; an audit_events row with the acting user exists after issue, next, skip and cancel; get_token_view has no personal fields, reports expired by date, returns nothing for an unknown code; has_function_privilege shows anon can execute only get_token_view; staff of another organization gets NOT_FOUND or FORBIDDEN.
Jest integration test tests/integration/concurrency.test.ts against local Supabase (script: npm run test:integration): 20 parallel issue_token calls with different request_ids give unique numbers 1-20; 20 parallel calls with the same request_id return one token; 2 parallel advance_queue calls with the same expected_serving_token_id give exactly one success and one STALE_STATE.

```
Run: supabase db reset, npm run test:db, npm run test:integration, npm run
lint, npm run typecheck. Re-run scripts/realtime-spike.ts to confirm a
broadcast arrives after advance_queue.
Acceptance: every test passes; every deviation is listed.
```

Prompt 4 – Authentication, access control and admin

```
PHASE 4 - AUTHENTICATION, ACCESS CONTROL AND ADMIN
Read first: blueprint Sections 5 (FR-01 to 04, FR-16), 10 (SEC-01, 02, 07,
09, 15), 11 (F-01, F-02), 12, 15 (admin endpoints), DEC-12, DEC-14.

CURRENT STATE
Phases 1-3 are complete.

OBJECTIVE (in scope)
1. Supabase clients: src/lib/supabase/browser.ts, server.ts (cookie-based,
@supabase/ssr), service.ts (service role, import 'server-only').
2. src/middleware.ts: refresh the session; protect /dashboard and /reports
for staff and /admin for admin; unauthenticated users go to /login; a wrong
role sees a 403 page.
3. Extend the handler wrapper with withAuth({ role: 'staff' | 'admin' }):
load the profile; 401 UNAUTHENTICATED without a session; 403 FORBIDDEN for a
missing, inactive or wrong-role profile or an inactive organization; pass {
user, profile, supabase } to the handler. Check the Origin header on POST,
PATCH and DELETE (SEC-09).
4. Pages in English: /login (email, password, ONE generic error message),
/reset (request a link), /reset/confirm (new password, minimum 10
characters), logout. Build Button, TextField and Banner in src/components/ui.
5. supabase/config.toml: sign-ups disabled, minimum password length 10. Write
the same settings into docs/deployment.md as a to-do for the cloud project.
6. Admin API as in Section 15: GET and POST /api/admin/organizations, PATCH
/api/admin/organizations/[id], GET and POST /api/admin/staff, PATCH
/api/admin/staff/[id], GET /api/admin/overview, backed by a new SECURITY
DEFINER function admin_overview() that only admin can execute and that
returns counts only (add it in a migration with a pgTAP test; admin has no
direct read policy on tokens). Staff creation uses the Auth admin API through
the service client: email_confirm true, a random temporary password returned
ONCE to the admin and never stored or logged, profile created in the same
server operation; if the profile insert fails, delete the auth user.
Deactivation sets is_active false; deactivated users are rejected immediately
by withAuth and RLS.
7. Admin pages: /admin overview counts, organizations list and form, staff
list and form (temporary password shown once, with a copy button).
```

```
8. ESLint no-restricted-imports rule so that service.ts can be imported only
from the four allowed server modules.

OUT OF SCOPE
Queue and token features, visitor page, notifications.

VERIFICATION AND ACCEPTANCE
Jest: withAuth cases (no session, inactive profile, wrong role, inactive
organization, origin mismatch) with mocked clients; Zod schemas. Postman
collection in tests/api with every admin endpoint, success and each
documented error. Playwright: staff logs in; wrong password shows the one
generic error; unauthenticated /dashboard redirects to /login; staff cannot
open /admin; admin creates an organization and a staff account; that staff
logs in with the temporary password; after deactivation the staff cannot log
in or act.
Run: npm run lint, npm run typecheck, npm test, npm run test:db, npm run
test:e2e, npm run build.
Acceptance: FR-01 to 04 and FR-16 criteria met; SEC-01, SEC-02 and SEC-15
covered by the tests above; the lint rule blocks a test import of service.ts
from a client file.
```

Prompt 5 – Staff controller

```
PHASE 5 - STAFF CONTROLLER
Read first: blueprint Sections 5 (FR-05 to 09, 13, 14), 9, 11 (F-03 to F-05,
F-08), 12, 13, 15 (queue endpoints), DEC-15.

CURRENT STATE
Phases 1-4 are complete: database and functions, auth, admin.

OBJECTIVE (in scope)
1. Migration: an AFTER INSERT trigger on queues (SECURITY DEFINER) that
creates the queue's queue_state row (service_date today, avg_service_minutes
= default_service_minutes, status active). Phase 2 seed rows stay valid.
2. Handlers from Section 15 using withAuth({ role: 'staff' }) and the user's
own session: POST /api/queues, PATCH /api/queues/[id], GET
/api/queues/[id]/dashboard, POST /api/queues/[id]/tokens, POST
/api/queues/[id]/advance, POST /api/queues/[id]/tokens/[tokenId]/cancel. Map
each database exception message to its API code and HTTP status. A
STALE_STATE response includes the current queue state (read from
queue_state). The dashboard returns state, serving token, next 5 waiting
tokens with the phone masked (show only the last 3 digits) and today's
counts. After advance and cancel, the handler does not yet process
```

```
notifications (Phase 7).
3. src/features/queue/schemas.ts: phone accepts 01XXXXXXXXX, +8801XXXXXXXXX
or 8801XXXXXXXXX (operator digit 3-9) and normalises to +8801XXXXXXXXX,
rejects everything else; queue name 2-60; default minutes 1-120; request_id
is a UUID.
4. Pages: /dashboard (queue list with serving number, waiting count and
active flag, plus a create-queue form) and /dashboard/queues/[id]
(controller): very large serving number; Next and Skip buttons that send
expected_serving_token_id and are disabled while a request runs; STALE_STATE
refreshes the screen and shows "Someone already advanced the queue"; a
network error shows "Not saved" with Retry, reusing the same request; an
issue-token form (optional phone; the client generates request_id once per
submission and reuses it on retry); the next 5 waiting tokens, each with
Cancel behind a ConfirmDialog; current average minutes; counts. Until Phase 6
the screen refreshes from action responses, on window focus and every 10
seconds.
5. Token slip shown right after issuing: queue name, number, date, a QR code
of {origin}/t/{public_code} (qrcode.react), the short link as text, and a
Print button with print CSS that prints only the slip.
6. Add to src/components/ui: Table, StatusBadge, ConfirmDialog, BigNumber,
QRCard.
7. Accessibility: labels on every input, visible focus, 44 px touch targets,
aria-live on the serving number.

OUT OF SCOPE
Visitor page, realtime hook, notifications, reports, pause and recall.

VERIFICATION AND ACCEPTANCE
Jest: schemas (all phone variants and rejections), error mapping, masking.
Postman: every endpoint, success and each documented error. Playwright:
create a queue; issue tokens 1-3; the slip shows the number and a QR; press
Next three times and check statuses; in a second browser context a stale
advance shows the conflict message; cancel a waiting token through the
confirmation.
Run: npm run lint, npm run typecheck, npm test, npm run test:db, npm run
test:e2e, npm run build.
Acceptance: FR-05 to 09, 13, 14 criteria met; two staff pressing Next
together never skip or repeat a token (Phase 3 concurrency test plus the
Playwright conflict test).
```

Prompt 6 – Visitor live page and realtime

PHASE 6 - VISITOR LIVE PAGE AND REALTIME
Read first: blueprint Sections 5 (FR-10 to 13, NFR-01, 02, 05), 9, 10 (SEC-04), 11 (F-06, F-07), 12, 13, 15 (get_token_view), DEC-03, DEC-09.

CURRENT STATE
Phases 1-5 are complete.

OBJECTIVE (in scope)
1. src/features/queue/eta.ts: peopleAhead(waitingNumbers, servingNumber, myNumber) = (servingNumber is not null and lower than myNumber ? 1 : 0) + the count of waitingNumbers lower than myNumber; etaMinutes(peopleAhead, avgMinutes) = Math.round(peopleAhead * avgMinutes), never below 0.
2. src/features/live: a useQueueLive(queueId) hook that subscribes to the public topic queue:<id> with the anon client, applies a message only when its version is higher than the current one, and loads fresh state on SUBSCRIBED. If the channel errors or closes it reports "reconnecting", polls get_token_view every 10 seconds and retries the subscription with backoff.
3. /t/[code] (client-first, mobile-first from 360 px, metadata noindex): calls get_token_view; shows the serving number (64 px, bold), my number, people ahead and "about N minutes"; status text for waiting, serving ("Your turn"), completed, skipped ("You were skipped, please contact the desk"), cancelled, expired, paused (no ETA), not found, loading and reconnecting.
4. Language: Bangla by default with an English toggle; dictionaries in src/i18n; the choice is kept in localStorage inside try/catch and the page works when storage fails; Noto Sans Bengali through next/font.
5. In-page near-turn alert: when peopleAhead first becomes 3 or fewer for a waiting token, show a Banner and call navigator.vibrate when available; remember it in localStorage by public code (try/catch) so a refresh does not repeat it. The banner must not depend on sound.
6. An aria-live region announces status changes; status is never shown by colour alone.
7. Switch the staff controller (Phase 5) to useQueueLive and keep the 10-second polling as a fallback.
8. The visitor page must never request or render a phone number, token ID or organization.

OUT OF SCOPE
Web Push, SMS, reports.

VERIFICATION AND ACCEPTANCE
Jest on eta.ts: serving 12, mine 18, waiting 13-17 gives 6 ahead and 42 minutes at 7 minutes average; cancelled gaps (serving 12, waiting [13,15,17], mine 18) give 4 ahead; unsorted arrays; serving equal to mine gives 0 ahead;

```
serving higher than mine adds nothing; rounding of a 0.5 average. Hook test:
an older version message is ignored.
Playwright with two browser contexts: staff presses Next and the visitor page
shows the new serving number within 2 seconds (record the measured time);
setOffline(true) shows "Reconnecting" and setOffline(false) recovers; the
alert banner appears once at 3 ahead and not after a reload; responses seen
by the visitor page contain no phone, token ID or organization.
Measure first content of the visitor page on emulated 4G and report the
number.
Run: npm run lint, npm run typecheck, npm test, npm run test:e2e, npm run
build.
Acceptance: FR-10, FR-11 (display), FR-12 (in-page part), FR-13 messages;
SEC-04; measured NFR-01 and NFR-02 numbers reported honestly, even if they
miss the target.
```

Prompt 7 – Notifications and PII purge

```
PHASE 7 - NOTIFICATIONS AND PII PURGE
Read first: blueprint Sections 5 (FR-12, FR-17), 9 (transactions and failure
handling), 10 (SEC-07, 08, 11, 12), 11 (F-07, F-10), 14 (retention), 15, DEC-
12, DEC-13, DEC-16.

CURRENT STATE
Phases 1-6 are complete. The outbox table and enqueue_near_turn exist;
nothing sends yet.

OBJECTIVE (in scope)
1. Migration: register_push(p_code, p_endpoint, p_p256dh, p_auth) per Section
15 (service role only; the token must be waiting or serving; upsert by
endpoint; then calls enqueue_near_turn) and purge_expired_pii() (clear phone
and delete push_subscriptions and outbox rows for tokens whose service_date
is older than 7 days; delete audit_events older than 180 days; return the
counts). EXECUTE on both revoked from anon and authenticated.
2. public/sw.js: handle push (show the notification from the payload) and
notificationclick (focus or open the token page).
3. "Alert me" button on the visitor page: asks for permission only after a
tap; subscribes with NEXT_PUBLIC_VAPID_PUBLIC_KEY; posts to
/api/public/tokens/[code]/push; clear states for unsupported browser,
permission denied (explain, and mention SMS when a phone was given) and
subscribed. Add a plain-text note that on iPhone notifications may need the
page added to the Home Screen; do not claim certainty.
4. POST /api/public/tokens/[code]/push: Zod validation; best-effort rate
limit (in memory per instance, for example 10 requests per minute per IP and
```

```
code; document it as best effort on serverless); calls register_push through
the service client.
5. src/features/notifications/outbox.ts: processOutbox(queueId) selects up to
20 pending rows for tokens of that queue (service client). Push rows are sent
with the web-push package (state why the dependency is needed; VAPID values
from env). SMS rows go through an SmsProvider interface send(to, text)
returning { ok, error? }, with a MockSmsProvider that logs only a masked
number, selected by SMS_PROVIDER=mock. On success set sent and sent_at; on
failure increase attempts and set failed after 3; delete the subscription
when the push service answers 404 or 410; never put a phone number in
last_error or in logs. Message text is short, in Bangla and English, for
example "3 people ahead of you (serial N)".
6. Call processOutbox after the issue, advance and cancel handlers have
committed. A failure is logged with the requestId and never changes the HTTP
result of the queue action.
7. Purge scheduling: first check, in current documentation, whether pg_cron
is available on the target Supabase plan. If yes, schedule
purge_expired_pii() daily. If not, add POST /api/internal/purge protected by
a CRON_SECRET header plus a Vercel Cron entry (check the plan's cron limits).
Record the choice in docs/decisions.md.
8. A real SMS gateway is NOT part of this phase (decision D-02 is open): keep
the interface and the mock only. FR-17 stays "partially implemented".

OUT OF SCOPE
Real SMS provider, reports.

VERIFICATION AND ACCEPTANCE
Jest on processOutbox with a fake push sender and fake SMS provider: success;
failure then retry; third failure gives failed; 404 and 410 remove the
subscription; no phone number in logs (spy on the logger); at most 20 rows
per run. Jest on the rate limiter and the push route (validation and errors).
pgTAP: register_push accepts only waiting or serving tokens and cannot be
executed by anon or authenticated; purge_expired_pii clears phone and
subscriptions older than 7 days and keeps token rows.
Write docs/manual-tests.md with a real-device push test on Android Chrome
(permission flow, notification at 3 ahead, click opens the page) and mark it
"Not run" until a person executes it.
Run: npm run lint, npm run typecheck, npm test, npm run test:db, npm run
test:integration, npm run build.
Acceptance: FR-12 complete (in-page and push); FR-17 with the mock only; SEC-
08 (best effort), SEC-11, SEC-12 covered.
```

Prompt 8 – Reports and Should-have features

PHASE 8 - REPORTS AND SHOULD-HAVE FEATURES
Read first: blueprint Sections 5 (FR-15, 18, 19), 9 (other workflows), 11 (F-09, F-11, F-12), 14, 15, DEC-17.

CURRENT STATE
Phases 1-7 are complete.

OBJECTIVE (in scope)
Do the MUST item first, then the SHOULD items in order, one commit per item. Stop the SHOULD items if time is short and say so in the handoff.
1. MUST - report_summary(p_queue_id, p_from, p_to): callable by staff of the queue's organization and by admin; a range over 31 days raises VALIDATION_FAILED; returns counts by status, average wait seconds (called_at minus issued_at for tokens that were called), average service seconds (completed tokens) and no phone numbers. Admin may pass a null queue for all queues. GET /api/reports and the page /dashboard/reports (queue select, date range, totals, averages, a daily table, an empty state "No data for this period").
2. SHOULD F-11 - pause_queue, resume_queue and recall_token with audit events, endpoints and controller UI. Pause sets status paused and paused_at, blocks Next and issuing (QUEUE_PAUSED), and the visitor page shows "paused" with no ETA. Resume moves the serving token's called_at forward by the paused interval so a pause never inflates service time. Recall works only for a skipped token (TOKEN_NOT_SKIPPED otherwise) and only once per token (ALREADY_RECALLED); the token returns to waiting and, having the lowest waiting number, is called next.
3. SHOULD F-12 - hourly counts (Asia/Dhaka hours) in report_summary and GET /api/reports/export as CSV. Neutralize formula injection by prefixing any cell that starts with =, +, - or @ with a single quote. No phone numbers in the export.

OUT OF SCOPE
Real SMS provider, other report types.

VERIFICATION AND ACCEPTANCE
pgTAP with a seeded dataset of known tokens and timestamps: exact counts and averages; a range over 31 days rejected; staff of another organization rejected; admin sees all; pause blocks Next and issue; after resume the next service_seconds excludes the paused time; recall rules; hourly counts. Jest: CSV escaping and formula neutralization. Playwright: the report page shows the seeded figures; pausing shows the paused state on the visitor page.

```
Run: npm run lint, npm run typecheck, npm test, npm run test:db, npm run
test:e2e, npm run build.
Acceptance: FR-15 met. FR-18 and FR-19 are met only if implemented; otherwise
the handoff marks them "not implemented".
```

Prompt 9 – Security hardening and regression

```
PHASE 9 - SECURITY HARDENING AND REGRESSION
Read first: blueprint Sections 5 (NFR), 10 (all SEC items), 13
(accessibility), 14 (RLS), 15.

CURRENT STATE
Phases 1-8 are complete. This phase adds no features. Fix only what the
checks reveal, with minimal changes.

OBJECTIVE (in scope)
1. Content-Security-Policy and the final security headers in next.config:
allow only self, the Supabase URL and its wss endpoint; use nonces or hashes
if Next.js requires them; confirm the app works in dev and in the production
build.
2. A security test matrix (pgTAP and Jest): loop over every table and prove
that anon and cross-organization access is denied; has_function_privilege
checks for every function (anon may execute only get_token_view;
authenticated only the staff functions; register_push and purge_expired_pii
only the service role).
3. A check that src/lib/supabase/service.ts is not imported by client code,
and a grep of the production build for a service-role key sentinel.
4. Origin-check tests on every state-changing handler; tests that force
errors and assert responses never contain SQL text, stack traces or phone
numbers.
5. Dependencies and secrets: npm audit (report high and critical findings), a
Dependabot configuration, and a secret scan with gitleaks or an equivalent
(report the result).
6. Accessibility: @axe-core/playwright on login, controller, slip, visitor
page (every reachable state) and admin; fix serious and critical findings;
calculate the contrast ratio of every palette colour pair and report the
numbers.
7. Evidence for NFR-01 and NFR-02: scripts/load/realtime-latency.ts with 100
anonymous subscribers on one queue measuring the time from Next to delivery
over 20 advances (report p50 and p95); a Lighthouse or Playwright trace of
the visitor page on emulated 4G (report the number).
8. Full regression: lint, typecheck, unit, pgTAP, integration, Playwright,
build. Also extend the GitHub Actions workflow with supabase start plus
```

```
test:db, test:integration and test:e2e jobs, and report coverage for the
TypeScript business-logic modules (target 70%, NFR-08).

OUT OF SCOPE
New features and refactors unrelated to findings.

VERIFICATION AND ACCEPTANCE
Acceptance: no open high or critical vulnerability and no axe serious or
critical issue; NFR-01 and NFR-02 results reported exactly as measured (if a
target is missed, state the gap and the options; do not change the test to
pass); every suite passes.
```

Prompt 10 – Deployment and release

```
PHASE 10 - DEPLOYMENT AND RELEASE
Read first: blueprint Sections 6 (environments and secrets), 10 (SEC-07, SEC-
14), 14 (retention), docs/decisions.md, DEC-16, assumption A-04.

CURRENT STATE
Phases 1-9 are complete. You may not have access to Vercel or Supabase
accounts: do everything that can be done in the repository, and mark each
step a human must perform.

OBJECTIVE (in scope)
1. docs/deployment.md runbook: create the Supabase cloud project; link it and
run supabase db push; Auth settings (sign-ups disabled, minimum password
length 10, site URL and redirect URLs, SMTP for reset emails, noting the
default email limits after checking current documentation); realtime settings
for public topics as found in Phase 2; create the first admin account (script
or dashboard) and store its password in a password manager; Vercel project
and environment variables per environment (from .env.example); VAPID key
generation command; CRON_SECRET; purge scheduling as chosen in Phase 7;
custom domain with HTTPS (required for Web Push).
2. From current official documentation, record with links the free-plan
limits that matter here (realtime concurrent connections and messages,
database size, project pausing on inactivity, cron availability, email
limits). Compare them with NFR-01 and the demo plan and flag risks
(assumption A-04).
3. /api/health also checks that the database is reachable (select 1 through
the server client) and never leaks details.
4. Rollback in the runbook: redeploy the previous Vercel deployment; database
changes follow a forward-fix policy; take an export (pg_dump or the plan's
backup) before applying migrations.
```

```
5. Re-run scripts/load/realtime-latency.ts against the deployed project (with
fewer subscribers if the plan limits connections) and report the numbers. A
production smoke test in tests/smoke (Playwright, mostly read-only) and a
checklist: login, create queue, issue token, open the visitor page, Next, see
the update, report page, logout; plus the manual push test.
6. A release checklist in the README. Tag v1.0.0 only when every Phase 9
acceptance item is met.

OUT OF SCOPE
New features and monitoring beyond Vercel and Supabase logs.

VERIFICATION AND ACCEPTANCE
Run: npm run lint, npm run typecheck, npm test, npm run build; run the smoke
test against the deployed URL if a deployment was made, otherwise mark it
"Not run".
Acceptance: the runbook is complete and consistent with the repository; the
list of human-only steps shows the status of each; any unverified limit is
marked unverified.
```

19. Software Quality Assurance

Testing is built into each phase prompt. This section lists the strategy, the critical test cases and the release rules. **No test has been run:** every Actual Result is empty and every Status is "Not run" until someone executes it.

Strategy

Level	Tool	Covers	Phases	Gate
Unit (TypeScript)	Jest	`eta.ts`, Zod schemas, error mapping, handler wrapper, outbox, CSV	1–9	70% or more coverage on TypeScript business-logic modules
Database	pgTAP (`supabase test db`)	Constraints, RLS, functions, privileges, purge, reports	2–9	All pass
Integration	Jest against local Supabase	Concurrency, broadcast delivery	3, 7	All pass

Level	Tool	Covers	Phases	Gate
API	Postman collection (Postman or Newman)	Every endpoint, success and documented errors	4–8	All pass
End-to-end	Playwright	Critical journeys, two-browser realtime, offline recovery	4–10	All pass
Accessibility and responsive	axe through Playwright, viewports 360 / 768 / 1280	All screens and states	9	No serious or critical issue
Security	Grants matrix, headers, `npm audit`, secret scan	Section 10 controls	9	No open high or critical finding
Performance	Load script, Lighthouse or Playwright trace	NFR-01, NFR-02	9	Targets met, or gap accepted in writing
Build and deploy	Build, smoke test	Release candidate	10	Smoke test passes
Manual	Real phone, browsers, outage drill, backup drill	Push, NFR-03, NFR-06, SEC-14	7, 9, 10	Recorded result

Critical test cases: functional

Test ID	Requirement	Scenario	Preconditions	Steps	Expected result	Actual result	Status
TC-01	FR-01	Staff signs in	Active staff account	Open `/login`, enter valid email and password, reload, log out	Goes to `/dashboard`; session survives reload; logout ends it	—	Not run

Test ID	Requirement	Scenario	Preconditions	Steps	Expected result	Actual result	Status
TC-02	FR-01	Wrong credentials	Active staff account	Submit a wrong password, then an unknown email	Same generic error both times	—	Not run
TC-03	FR-02	Reset link is single use	Active staff account	Request reset, open link, set password, open the same link again	New password works; second use and expired link are refused	—	Not run
TC-04	FR-03	Organization deactivated	Organization with staff and a queue	Admin deactivates it; staff presses Next	Refused with FORBIDDEN; visitors can still read their tokens	—	Not run
TC-05	FR-04	Staff isolation	Organizations A and B with queues and tokens	Staff A lists queues and requests B's queue and token by ID	Only A's data listed; B's resources return 404	—	Not run
TC-06	FR-05	Queue validation	Staff signed in	Create queues with 1-character and 61-character names, minutes 0 and 121, a duplicate name	Each rejected with a field error; a valid queue is created	—	Not run
TC-07	FR-06	Concurrent issue	Empty active queue	20 parallel issue requests with different request IDs	Numbers 1–20, no duplicate, no gap	—	Not run
TC-08	FR-06	Retry after network error	Active queue	Send issue twice with the same request ID	Same token returned; last issued number increased once	—	Not run
TC-09	FR-07	Slip and QR	Token issued	Open slip, scan QR with a phone, press Print	QR opens that token's live page; print shows the slip only	—	Not run
TC-10	FR-08	Next and concurrent Next	Serving #1, waiting #2 and #3	Press Next; then two clients send Next with the same expected token	#1 completed, #2 serving; the second concurrent request gets STALE_STATE; no token skipped or repeated	—	Not run
TC-11	FR-09	Skip excluded from average	Three completed tokens of 5, 6 and 7 minutes	Skip the serving token and read the average	Average stays 6.0	—	Not run

Test ID	Requirement	Scenario	Preconditions	Steps	Expected result	Actual result	Status
TC-12	FR-10	Live update within 2 seconds	Visitor page and staff controller open	Staff presses Next	Visitor shows the new serving number within 2 seconds (record the time)	—	Not run
TC-13	FR-10	Offline recovery	Visitor page open	Go offline 30 seconds, staff presses Next, go online	"Reconnecting" shown; correct state appears after reconnecting	—	Not run
TC-14	FR-11	Worked example	Serving #12, waiting #13–#17, mine #18, average 7.0	Open the visitor page	6 ahead and "about 42 minutes"	—	Not run
TC-15	FR-11	Average rule cases	Queue default 5 minutes	Complete 0, 1, 3, 5 and 7 tokens in separate runs; add one 4000-second and one 10-second service	0 gives 5.0; others use up to the last 5; the 4000-second and the 10-second services are excluded	—	Not run
TC-16	FR-12	Near-turn alert once	Waiting #1–#6	Visitor of #6 watches; staff presses Next 3 times; reload	Banner at 3 ahead, once; none after reload; one outbox row per channel	—	Not run
TC-17	FR-12	Web Push on a real phone (manual)	HTTPS deployment, Android Chrome	Tap "Alert me", allow, advance to 3 ahead, tap the notification	Notification arrives; tap opens the token page	—	Not run
TC-18	FR-13	State messages	Tokens in each state	Open pages for skipped, cancelled, completed, expired, paused	Each shows its defined message; no ETA when paused or ended	—	Not run
TC-19	FR-14	Dashboard consistency	Several actions done	After each action compare the dashboard with the database	Serving, next 5, counts and average match	—	Not run
TC-20	FR-15	Report figures	Seeded dataset with known numbers	Request the seed range; request 32 days	Figures equal the known values; 32 days refused	—	Not run
TC-21	FR-16	Admin overview	Admin and staff accounts	Open `/admin` as each	Admin sees counts; staff gets 403	—	Not run
TC-22	FR-17 (Should)	SMS retry with mock	Token with phone; failing mock provider	Trigger near-turn	3 attempts, then `failed`; no phone number in logs	—	Not run

Test ID	Requirement	Scenario	Preconditions	Steps	Expected result	Actual result	Status
TC-23	FR-18 (Should)	Pause, resume, recall	Serving token and a skipped token	Pause 10 minutes, resume, complete; recall the skipped token twice	Service time excludes the pause; second recall gives ALREADY_RECALLED	—	Not run
TC-24	FR-19 (Should)	Hourly counts and CSV	Seeded dataset	Export CSV with a queue name starting with `=`	Hourly counts match; formula neutralized; no phone numbers	—	Not run

Critical test cases: non-functional

Test ID	Requirement	Scenario	Preconditions	Steps	Expected result	Actual result	Status
TC-25	NFR-01	100 viewers	100 subscribers on one queue	20 advances; record delivery times	p95 within 2 seconds (report p50 and p95)	—	Not run
TC-26	NFR-02	First content on 4G	Production build, emulated 4G, mid-range phone profile	Load `/t/{code}`	First useful content within 3 seconds	—	Not run
TC-27	NFR-03	Outage drill (manual)	Running system	Cut the app's network, staff continue on paper, restore	Staff screen shows an offline banner; no data corrupted; state recovered with the LIM-01 procedure	—	Not run
TC-28	NFR-04	Responsive layout	Built app	Open live page, slip and controller at 360, 768 and 1280 px	No horizontal scroll (tables excepted); controls usable	—	Not run
TC-29	NFR-05	Accessibility	All screens and states	Run axe; keyboard-only run; compute contrast ratios	No serious or critical finding; every action by keyboard; text contrast 4.5:1 or better	—	Not run
TC-30	NFR-06	Browser support (manual)	Latest 2 versions of Chrome, Safari, Firefox, Edge	Run the smoke journey on each	Journey completes; differences recorded	—	Not run

Test ID	Requirement	Scenario	Preconditions	Steps	Expected result	Actual result	Status
TC-31	NFR-07, SEC-11	PII purge	Tokens with phones and subscriptions older and newer than 7 days	Run `purge_expired_pii()`	Old phones and subscriptions removed; token rows kept; newer data untouched	—	Not run
TC-32	NFR-08	Code quality gates	CI	Run lint, typecheck, Jest with coverage	Clean; 70% or more on TypeScript business-logic modules	—	Not run
TC-33	NFR-09	Dhaka day boundary	Queue state dated yesterday	Issue tokens across the rollover	Numbering restarts; leftovers `expired`; timestamps stored in UTC	—	Not run

Critical test cases: security

Test ID	Requirement	Scenario	Preconditions	Steps	Expected result	Actual result	Status
TC-34	SEC-01	Sign-up and password policy	Auth settings applied	Attempt public sign-up; set a 9-character password	Both refused	—	Not run
TC-35	SEC-02, 03, 15	RLS and grants matrix	Seed data	For every table and function, test as anon, staff A, staff B, admin and deactivated staff	Only the access in Sections 14 and 15 works; deactivated staff are rejected at once	—	Not run
TC-36	SEC-03, 04	Public view privacy	Token with a phone	Call `get_token_view`; inspect payload and broadcast message; try unknown codes	No phone, token ID or organization; unknown code returns nothing; code is 22 random characters	—	Not run
TC-37	SEC-05, 06	Validation and injection	Handlers running	Send oversized, wrongly typed and SQL-like input, invalid phone and status	400 VALIDATION_FAILED; strings treated as data; nothing executes	—	Not run
TC-38	SEC-07	Secrets	Production build and repository	Search the build output for the service key; run the secret scan	Nothing found	—	Not run

Test ID	Requirement	Scenario	Preconditions	Steps	Expected result	Actual result	Status
TC-39	SEC-08	Rate limit (best effort)	Push endpoint	Send 20 requests in one minute	Excess requests get 429 on one instance; result documented as best effort	—	Not run
TC-40	SEC-09	Origin check and headers	Handlers and pages	POST with a foreign Origin; inspect response headers	Foreign origin refused; CSP and security headers present	—	Not run
TC-41	SEC-10	Audit events	Staff actions	Issue, Next, Skip, Cancel	One audit event each, with actor and time	—	Not run
TC-42	SEC-12	Clean errors	Forced failures	Force database and handler errors	Generic message and requestId only; no SQL, stack trace or phone in responses or logs	—	Not run
TC-43	SEC-13	Dependencies	CI	Run `npm audit`; confirm Dependabot is on	No unresolved high or critical finding	—	Not run
TC-44	SEC-14	Backup drill (manual)	Production-like project	Export, restore into a scratch project, run the smoke test	Restore succeeds; data intact	—	Not run

Release and defect rules

- **Blocks release:** any failing Must test (TC-01 to TC-21 except Should rows, and TC-28 to TC-44 except TC-39 which is best effort); any failing data-isolation test (TC-05, TC-35, TC-36); any open high or critical security finding; a failing build; migrations that do not apply cleanly to an empty database. A missed NFR-01 or NFR-02 target blocks release unless the team writes down the accepted gap.
- **Failed test:** log it with test ID, severity (blocker, major, minor), owner and fix. A test is never edited to pass without a written reason.
- **Retest:** after a fix, rerun the failed test and the whole suite of the affected phase; run the full suite before every release.
- Should rows (TC-22 to TC-24) block only the feature they cover, which is then marked "not implemented".

Release checklist

- ☐ All blocking tests pass, with Actual Result recorded in this table
- ☐ Lint, typecheck, build clean; coverage target met
- ☐ `npm audit` and secret scan clean
- ☐ Production environment variables set; service-role key only on the server
- ☐ Migrations applied; first admin account created; sign-ups disabled

- ☐ Smoke test passed on the deployed URL; manual push test done
- ☐ Backup taken before release; rollback steps read by the team
- ☐ Known limitations (Section 22) acknowledged

Definition of done (per feature): functionality implemented; acceptance criteria met; validation and permissions enforced; edge and error cases handled; tests written and run with results recorded; build, typecheck and lint pass; no regression in existing tests; docs and configuration updated; no known blocking issue hidden.

20. Deployment and Operations

Kept to what a reliable release needs: Vercel and Supabase cloud, no extra DevOps stack. The runbook itself is produced in Phase 10.

Environments

Setting	Local	Production
Database, Auth, Realtime	Supabase CLI in Docker	Supabase cloud project
App host	`next dev`	Vercel
Reset emails	Local test inbox from the Supabase CLI	SMTP configured; default email limits to be verified
Web Push	Development VAPID keys (localhost is allowed)	Separate VAPID keys; HTTPS domain required
SMS	Mock	Mock until D-02 is decided
Secrets	`.env.local`, git-ignored	Vercel environment variables; never in git
Purge	Called by hand	Scheduled as decided in Phase 7 (DEC-16)
Staging	Not planned	Optional second project if free limits allow

Deployment procedure

1. Create the Supabase project; set Auth options (sign-ups off, password length 10, site URL and redirect URLs).
2. Take an export of the database (even when empty, to practise).
3. Link the project and run `supabase db push` after reviewing the migrations.
4. Create the first admin account and store its password in a password manager.
5. Create the Vercel project; set environment variables from `.env.example`; connect the domain over HTTPS.
6. Deploy; open `/api/health`; confirm it reports the app and database as reachable.
7. Run the smoke test and the manual push test.
8. Tag the release.

Health, logging, monitoring

- `/api/health` returns a plain status, including a database check, and never reveals details.
- Server logs are structured lines with `requestId`, route, status and duration; they never hold phone numbers, tokens or keys (TC-42).
- Monitoring is Vercel and Supabase logs. An external uptime check is optional and not planned.

Backup and recovery: export before every migration and a weekly export during the demo period. Which backup features the free plan includes is not verified here; the Phase 10 prompt records it. Recovery is tested once (TC-44).

Rollback: redeploy the previous Vercel deployment. Database changes are fixed forward with a new migration; the pre-release export is the fallback.

Production smoke test: login, create queue, issue token, open the visitor page, press Next, see the update, open the report, log out; then one real-phone push test.

21. Risk Register and Decision Log

Risk register (replaces the table in Section 3; the new risks are R-08 to R-14)

ID	Risk	Probability	Impact	Mitigation	Phase
R-01	Concurrent clicks skip or duplicate a token	Medium	High	Row lock, expected-token check; TC-07, TC-10	3
R-02	Realtime drops on weak mobile networks	High	Medium	Reconnect and 10-second polling; TC-13	6

ID	Risk	Probability	Impact	Mitigation	Phase
R-03	Web Push not delivered (iPhone, permission denied)	High	Medium	In-page alert, SMS option, clear message	6, 7
R-04	SMS cost or gateway approval delays	Medium	Low	Mock provider; FR-17 is Should	7
R-05	Public page leaks data	Low	High	Broadcast topic and token view only; TC-36	2, 3, 9
R-06	Inaccurate ETA reduces trust	Medium	Medium	FR-11 rules, "about" wording, simulated test	3
R-07	Scope growth in 10 weeks with 3 students	Medium	High	Fixed MVP table; Should items last and cuttable	All
R-08	Free-tier limits or project pausing break the demo	Medium	High	Verify limits in Phase 10; keep the project active before the demo; paid plan as fallback	10
R-09	Anonymous public-topic broadcast does not work as assumed	Medium	High	Spike first in Phase 2; fallback is anonymous sign-in with private channels, or polling-only (10 seconds, which misses the 2-second target)	2
R-10	Staff do not press Next promptly, so ETA is wrong	High	Medium	Outlier cap, "about" wording, short staff guide; cannot be fixed by software	3, 10
R-11	AI-written code drifts from the blueprint	Medium	High	Shared Block, acceptance per phase, review of every handoff, tests	All
R-12	Day-rollover or time-zone bugs	Low	Medium	TC-33, Asia/Dhaka helper	3
R-13	A failed alert waits until the next staff action	Medium	Medium	Retry on next action; failed rows visible to staff	7
R-14	One deploy unit failing takes staff and visitors down together	Low	High	Degraded mode (NFR-03), health check, rollback	10

Decision log

ID	Decision	Rationale	Alternative considered
DEC-01	Modular monolith in Next.js	One deploy, one codebase	Separate API service
DEC-02	Queue changes inside Postgres functions	Row lock prevents races	Logic in handlers

ID	Decision	Rationale	Alternative considered
DEC-03	Broadcast from Database, public topic per queue	No table read; Supabase recommends it for scale	Postgres Changes
DEC-04	10-second polling fallback	Covers dropped sockets	Realtime only
DEC-05	Notification outbox	No lost alerts after a crash	Send inline only
DEC-06	Add Zod, Tailwind, Playwright	Validation, fast UI, end-to-end tests	Leave out
DEC-07	Jest, Postman, Playwright, pgTAP	Matches proposal plus database tests	Vitest
DEC-08	SECURITY DEFINER functions, no write policies	Single enforced write path	Invoker functions with write policies
DEC-09	`waiting_numbers` array in `queue_state`	Exact people ahead without reading tokens	Derive from counters
DEC-10	Status `expired`	Clean day rollover	Delete leftovers
DEC-11	Lazy day rollover	No scheduler needed	Midnight job
DEC-12	Service-role client in one file, four modules	Auditable use of the key	Wider use
DEC-13	Best-effort rate limit	Simple at this scale	Database-backed limiter
DEC-14	Temporary password shown once	No invitation email setup	Email invitations
DEC-15	Staff polling in Phase 5, realtime in Phase 6	Testable without the hook	Realtime first
DEC-16	Purge scheduler chosen in Phase 7	Availability unverified	Assume pg_cron

ID	Decision	Rationale	Alternative considered
DEC-17	New codes `TOKEN_NOT_SKIPPED`, `ALREADY_RECALLED`	Recall rules need clear messages	Reuse `TOKEN_NOT_WAITING`

22. Final Traceability and Readiness Review

This review cross-checks the PRD, feature specifications, architecture, schema, API contracts, phases and tests. Nothing is implemented or tested yet; every status below means "specified", not "working".

Inconsistencies found in the blueprint and what happened

ID	Finding	Resolution
RV-01	Original FR-11 left out the serving token, so the 42-minute example did not match	Fixed in Part 2: people ahead includes the serving token
RV-02	Part 1 had visitors reading `queue_state` anonymously	Replaced by Broadcast from Database (DEC-03); SEC-03 text and the TRD updated
RV-03	Five token states in Part 1, six in Part 3	`expired` added (DEC-10); FR-13 updated
RV-04	FR-16 admin overview needs counts, but admin has no read policy on tokens	Prompt 4 adds an `admin_overview()` function. **Documentation gap:** it is not yet in the Section 15 function table; Prompt 4 is authoritative until Phase 4 merges it
RV-05	After downtime, staff pressing Next to catch up would add near-zero service times to the average	New decision D-04 (below): durations under 20 seconds are excluded from the average; FR-11, Section 9 and Prompt 3 updated
RV-06	Section 7 diagram and trade-off table still described row changes on `queue_state`	Updated to the broadcast wording

ID	Finding	Resolution
RV-07	Recall needed error codes missing from Section 15	Added to the Section 15 table (DEC-17)

Decision added in this part

ID	Decision	Default applied
D-04 / DEC-18	A service duration under 20 seconds is not counted in the moving average. It is the signature of catch-up presses of Next after an outage or a forgotten click.	Applied now; change the 20-second value in FR-11 and Prompt 3 if you prefer another threshold

Traceability matrix: functional and non-functional requirements

Requirement	Feature / module	Design decision	Phase	Test cases	Status
FR-01	F-01 Sign-in	SEC-01	4	TC-01, TC-02	Defined
FR-02	F-01 Password reset	SEC-01	4	TC-03	Defined
FR-03	F-02 Organizations	DEC-08	4	TC-04	Defined
FR-04	F-02 Staff accounts	DEC-12, DEC-14	4	TC-05	Defined
FR-05	F-03 Queue setup	`queue_state` creation trigger (Prompt 5)	5	TC-06	Defined
FR-06	F-04 Issue token	DEC-02, DEC-08, DEC-11	3, 5	TC-07, TC-08	Defined
FR-07	F-04 Token slip	Sections 12–13	5	TC-09	Defined
FR-08	F-05 Next	DEC-02, DEC-08	3, 5	TC-10	Defined
FR-09	F-05 Skip	DEC-02	3, 5	TC-11	Defined
FR-10	F-06 Live view	DEC-03, DEC-04, DEC-09	6	TC-12, TC-13	Defined
FR-11	F-06 ETA	DEC-02, DEC-09, D-04	3, 6	TC-14, TC-15	Defined
FR-12	F-07 Near-turn alert	DEC-05, DEC-13	3, 6, 7	TC-16, TC-17	Defined

Requirement	Feature / module	Design decision	Phase	Test cases	Status
FR-13	F-05, F-06 States	DEC-10	3, 6	TC-18	Defined
FR-14	F-08 Staff dashboard	DEC-15	5, 6	TC-19	Defined
FR-15	F-09 Reports	`report_summary`	8	TC-20	Defined
FR-16	F-02 Admin overview	`admin_overview()` (Prompt 4)	4	TC-21	Defined
FR-17	F-10 SMS alert	DEC-05; provider open (D-02)	7	TC-22	Partially Defined
FR-18	F-11 Pause, resume, recall	Section 9	8	TC-23	Defined
FR-19	F-12 Peak hours, CSV	`report_summary`	8	TC-24	Defined
NFR-01	Realtime delivery	DEC-03	6, 9	TC-12, TC-25	Defined
NFR-02	Visitor page speed	Section 13	6, 9	TC-26	Defined
NFR-03	Degraded mode	Section 12; LIM-01	10	TC-27	Defined
NFR-04	Responsive layout	Section 13	5, 6, 9	TC-28	Defined
NFR-05	Accessibility	Section 13	9	TC-29	Defined
NFR-06	Browser support	none specific	9, 10	TC-30	Defined
NFR-07	Privacy	SEC-11	7	TC-31	Defined
NFR-08	Maintainability	Section 16	1, 9	TC-32	Defined
NFR-09	Time handling	DEC-11	3	TC-33	Defined

Traceability matrix: security controls

Control	Module	Design reference	Phase	Test cases	Status
SEC-01	Authentication	Auth settings	4	TC-01, TC-02, TC-34	Defined
SEC-02	Row Level Security	DEC-08, Section 14	2, 4	TC-05, TC-35	Defined
SEC-03	Public access	DEC-03	2, 3	TC-35, TC-36	Defined
SEC-04	Public code	`tokens.public_code`	3, 6	TC-36	Defined
SEC-05	Input validation	Zod, constraints	2, 5	TC-37	Defined
SEC-06	SQL injection	RPC only, fixed `search_path`	3, 9	TC-37	Defined

Control	Module	Design reference	Phase	Test cases	Status
SEC-07	Secrets	DEC-12	1, 9	TC-38	Defined
SEC-08	Rate limiting	DEC-13	7	TC-39	Partially Defined
SEC-09	CSRF, XSS, headers	Origin check, CSP	4, 9	TC-40	Defined
SEC-10	Audit	`audit_events`	3	TC-41	Defined
SEC-11	Phone retention	`purge_expired_pii()`	7	TC-31	Defined
SEC-12	Clean errors	Handler wrapper	1, 9	TC-42	Defined
SEC-13	Dependencies	Dependabot, audit	9	TC-43	Defined
SEC-14	Backup	Runbook	10	TC-44	Defined
SEC-15	Deactivation	`current_org_id()`, `withAuth`	2, 4	TC-35	Defined

Readiness table

Area	Status	Outstanding issue	Required action
Product scope and MVP	Defined	D-01 to D-03 unanswered (defaults apply)	Confirm or change the defaults
PRD	Defined	FR-17 depends on D-02 (SMS provider)	Choose a provider before building real SMS
TRD and stack	Defined	Free-tier limits unverified (A-04)	Verify in Phase 10; earlier if the demo is soon
Architecture	Partially Defined	Public-topic broadcast for anonymous clients unverified (R-09)	Phase 2 spike decides; fallback options listed
Detailed design	Defined	None known	None
Security and privacy	Partially Defined	SEC-08 limiter is best effort; iPhone push behavior unverified	Accept as limitations or add a database-backed limiter
Feature specifications	Defined	None known	None

Area	Status	Outstanding issue	Required action
Flows and UI/UX	Defined	No mockups exist; screens are specified in words only	Build in Phases 5–6; optional sketch first
Database schema	Defined	Not applied or tested	Phase 2
API contracts	Partially Defined	`admin_overview()` missing from Section 15 (RV-04); SMS gateway API unknown	Merge in Phase 4; verify gateway docs before FR-17
Project structure	Defined	None known	None
Implementation plan and prompts	Defined	Prompts not yet run	Start with Prompt 1
SQA	Defined	All 44 test cases "Not run"	Execute per phase and record results
Deployment	Partially Defined	No accounts or environments exist; runbook is written in Phase 10	Create Supabase and Vercel accounts
Implementation	Not started	Everything	Phases 1–10
Verification	Not started	Everything	Section 19

Scope realism: the Must scope is 10 phases in 10 weeks for 3 students. Phase 3 (concurrency-safe SQL), Phase 6 (realtime page) and Phase 7 (push, outbox) are the heaviest. This is a judgment, not an estimate backed by data. If time runs short, cut in this order: FR-19, FR-18, the Web Push part of FR-12 (keep the in-page alert), then the Bangla/English toggle (keep Bangla). Do not cut tests or the security controls.

Known limitations (to repeat in the README)

ID	Limitation
LIM-01	There is no "set current number" tool. After an outage, staff press Next once per patient already served on paper; with D-04 these quick presses do not distort the average. Do not use Skip to catch up, because visitors would see "skipped".

ID	Limitation
LIM-02	A failed alert is retried only on the next staff action (Section 9).
LIM-03	The push-subscribe rate limit is best effort.
LIM-04	Web Push on iPhone may need the page added to the Home Screen (unverified); in-page alert and SMS are the fallbacks.
LIM-05	ETA is only as good as staff pressing Next at the real moment a visit starts.
LIM-06	SMS runs on a mock provider until a gateway is chosen.

Your next actions

1. Answer D-01 (visitor language), D-02 (SMS gateway) and D-03 (recall of skipped tokens), or accept the defaults.
2. Save this blueprint as `docs/blueprint.md` in the repository.
3. Create the Supabase and Vercel accounts.
4. Run Prompt 1 (Section 18) with the Shared Block, review the handoff, then continue phase by phase.